



Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen Welten- Generation

Bachelorarbeit zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc) in Informatik

Eingereicht von

Maarten Behn
Fritz-Gansberg Str. 1, 28213 Bremen
Matrikelnummer: 6189259

Erstgutachter: Prof. Dr. Gabriel Zachmann

Zweitgutachter: Prof. Dr. Nico Hochgeschwender

AG CGVR

Fachbereich 3

Universität Bremen

Sommersemester 2026

Abgabetermin: 03.06.2026

1 Einleitung

Prozedurale Generierung ist ein etabliertes Verfahren zur automatischen Erstellung komplexer virtueller Welten, dass insbesondere in der Spieleentwicklung, Simulation und Computergrafik Anwendung findet. Dabei werden zunächst grobe Strukturen erzeugt, die anschließend schrittweise durch immer feinere Details ergänzt werden. Wird der Generationsalgorithmus in irgendeiner Form angepasst, kann es vorkommen, dass eine bereits erzeugte Welt nicht mehr dem aktuellen Stand des Algorithmus entspricht.

Die Rechenzeit eines prozeduralen Generationsalgorithmus steigt mit der Menge der erzeugten Details. Insbesondere bei komplexen Algorithmen kann die vollständige Neugenerierung einer Welt sehr zeitaufwendig sein. Dies stellt ein Problem dar, da in der Praxis Generationsalgorithmen häufig iterativ entwickelt und angepasst werden. Lange Neugenerationszeiten können hier sehr stören.

Ziel dieser Arbeit ist es, zu untersuchen, inwieweit sich die Neugenerationszeit einer prozeduralen Welt verkürzen lässt, wenn nur diejenigen Teile der Welt neu berechnet werden, die durch Änderungen am Generationsalgorithmus ungültig geworden sind.

Dafür wird ein System vorgestellt, welches einen Generationsalgorithmus als Abhängigkeits-Graph darstellt. Dieses verwendet Zwischenergebnisse für diejenigen Teile des Graphen, die sich nicht geändert haben, wieder.

Inhaltsverzeichnis

1	Einleitung	1
2	Stand der Technik	3
2.1	Noise & Zufälligkeit	3
2.2	Example based Model Synthesis	4
2.3	Graph Grammatiken	5
2.4	KI-basierte prozedurale Generation	6
2.5	Houdini & Blender Geometry Nodes	7
3	Theoretische Grundlagen	8
3.1	Minimales Berechnen (Lazy computation)	8
3.2	Graphen	8
3.3	Constructive Solid Geometry	8
3.4	Grafische-Programmierung	9
3.5	Violin Plot	10
4	Mein Lösungsansatz	11
4.1	Minimale Neuberechnung mit Model Synthesis	11
4.2	Abhängigkeits-Graph	12
4.3	Framework	12
4.4	Grafischer Editor	13
4.5	Template	14
4.6	Generator	16
4.7	Implementierung	20
4.8	Beispiele	22
5	Analyse	25
5.1	Theoretische Laufzeit	25
5.2	Reduktion der Neuberechnungszeit	26
5.3	Overhead	28
5.4	Nutzerfreundlichkeit	30
5.5	Erweiterbarkeit	30
6	Noch offene Fragestellungen	31
6.1	Nullwerte führen zu leeren Lösungen	31
6.2	Andere Datenstrukturen	31
6.3	Nutzerfreundlichkeitsstudie	31
6.4	Parallelisierung der Generierung	31
6.5	Automatische Cache-Optimierung	32
6.6	Skalierung auf sehr große Welten	32
6.7	Integration in bestehende Engines	32
6.8	Serialisierung und Persistenz	32
7	Fazit	33
8	Bibliografie	34

2 Stand der Technik

Dieses Kapitel gibt einen Überblick über bestehende Ansätze zur prozeduralen Generierung und bewertet, inwieweit sich diese zur minimalen Neuberechnung eignen.

Zunächst werden Noise-basierte Verfahren betrachtet. Diese sind in der Praxis weit verbreitet, bieten jedoch kaum Ansatzpunkte für gezielte Teilneuberechnungen. Graph-basierte Ansätze wie L-Systems haben aufgrund ihrer expliziten Regelstruktur zwar theoretisches Potenzial, konkrete Arbeiten zur minimalen Neuberechnungen existieren hier jedoch nicht. Anschließend werden KI-basierte Verfahren diskutiert und warum diese für minimale Neuberechnung grundsätzlich nicht geeignet sind. Houdini und Blender Geometry Nodes verfolgen einen ähnlichen Ansatz wie diese Arbeit, sind aber für einen anderen Anwendungskontext konzipiert.

2.1 Noise & Zufälligkeit

Viele prozedurale Generatoren nutzen Noise-Algorithmen, um Welten zu erzeugen. Die Grundidee ist, mehrere Noise-Funktionen wie Perlin-Noise oder Simplex-Noise mit unterschiedlichen Frequenzen und Amplituden zu überlagern. Ein Berg entsteht zum Beispiel durch eine Noise-Funktion mit niedriger Frequenz und hoher Amplitude, während kleinere Details wie Steine durch eine Funktion mit hoher Frequenz und niedriger Amplitude hinzugefügt werden. Die Struktur der Welt ergibt sich also nicht aus expliziten Regeln, sondern aus dem Zusammenspiel dieser Funktionen.

Der praktische Vorteil ist, dass sich dieser Ansatz leicht implementieren lässt und gut auf große Welten skaliert. Minecraft nutzt zum Beispiel Perlin-Noise, um sein Gelände und Höhlen effizient zu generieren.

Das Problem ist, dass sich mit Noise-Funktionen das Einhalten global geltender Regeln nur schwer garantieren lässt. Ob ein generiertes Gelände zum Beispiel immer einen zugänglichen Weg zwischen zwei Punkten hat, lässt sich aus den Noise-Funktionen allein



Abbildung 1: Beispiele für fehlerhaft generierte Welten in Minecraft (Azmy)

nicht bestimmen. Solche Regeln müssen in späteren Generationsschritten überprüft und die Welt gegebenenfalls angepasst werden. In vielen Generationssystemen werden diese Fehler toleriert, wenn sie nur selten auftauchen (Siehe Abbildung 1 für Beispiele von fehlerhafter Generation).

Für minimale Neuberechnungen sind Noise-basierte Verfahren eher ungeeignet, da theoretisch jede Noise-Ebene Einfluss auf jedes Ergebnis haben kann. Die Generierungslogik steckt implizit in der Komposition der Funktionen. Ändert man eine Funktion oder ihre Parameter, kann dies zu einer komplett anderen Welt führen.

2.2 Example based Model Synthesis

Eine relativ neuer Ansatz im Bereich der constraint-basierten Generation ist „Example-based model synthesis“ von Paul Merrell.

Die Grundidee ist hier, aus einem kleinen Eingabedatensatz eine größere Struktur zu erzeugen, die lokal denselben Regeln folgt.

Der Eingabedatensatz besteht aus einer Gitterstruktur, in der jeder Zelle ein Wert zugeordnet ist. Aus diesem Gitter wird eine Liste aller Nachbarkombinationen erstellt, die im Eingabedatensatz vorkommen.

Diese Liste enthält die Regeln, nach denen ein neues Modell erzeugt wird. Der Generationsalgorithmus läuft dabei wie folgendermaßen ab: Zunächst werden allen Zellen des neuen Gitters sämtliche möglichen Werte zugeordnet. Dann wird die Zelle mit den wenigsten verbleibenden Möglichkeiten ausgewählt und auf einen einzelnen Wert festgelegt. Anschließend werden aus den Nachbarzellen alle Werte entfernt, für die keine gültige Regel mehr existiert. Dieser Bereinigungsschritt wird rekursiv auf alle betroffenen Nachbarzellen ausgeweitet.

Der Vorgang wiederholt sich, bis jede Zelle genau einen Wert enthält. Da das Vorsehen, bestehende Model Synthesis Ereignisse an geänderte Regeln anzupassen, bisher unerforscht ist, habe ich es gleich von Beginn meiner Arbeit als Kernthema in Betracht

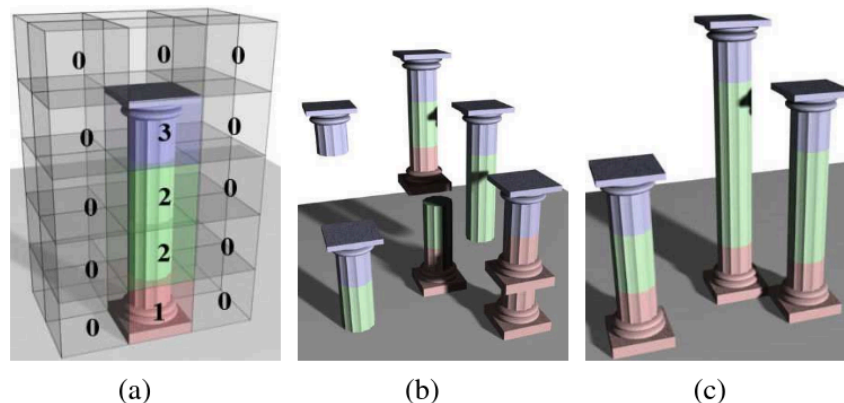


Abbildung 2: (a) A model composed of four model pieces, (b) An Inconsistent Model, (c) A Consistent Model (Abbildung aus Merrell „Example-based model synthesis“)

gezogen. In Abschnitt 4.1 werde ich darauf eingehen, warum ich diesen Ansatz verworfen habe.

2.3 Graph Grammatiken

Eine Graph-Grammatik ist ein System aus Regeln, die beschreiben, wie ein Graph verändert werden kann. Jede Regel besteht aus zwei Teilen: einem Teilgraph, der gesucht wird, und einem Teilgraph, der ihn ersetzt. Durch wiederholtes Anwenden solcher Regeln kann aus einem kleinen Startgraph eine komplexe Struktur wachsen. Zum Beispiel kann aus einem einzelnen Knoten durch eine Regel wie „Füge zwei Kindknoten hinzu“ ein ganzer Baum entstehen.

2.3.1 L-Systems

L-Systems sind eine frühe Anwendung dieser Idee, entwickelt in den 1960 Jahren vom Biologen Aristid Lindenmayer, um Pflanzenwachstum zu modellieren (Prusinkiewicz und Lindenmayer). Anstatt auf einem Graphen arbeiten sie auf einer Sequenz von Symbolen. In jedem Schritt werden alle Symbole gleichzeitig durch die passende Regel ersetzt. Ein Startsymbol F könnte die Regel haben „ersetze F durch $F[+F][-F]$ “, wobei $+$ und $-$ für eine Drehung stehen. Nach wenigen Iterationen entsteht so eine verzweigte Struktur, die wie ein Baum aussieht.

L-Systems eignen sich gut für Vegetation, weil natürliche Strukturen oft in sich selbst ähnlich sind. Ein Ast sieht aus wie ein kleiner Baum, ein Zweig wie ein kleiner Ast. Mit wenigen Regeln lassen sich so sehr organisch wirkende Formen erzeugen.

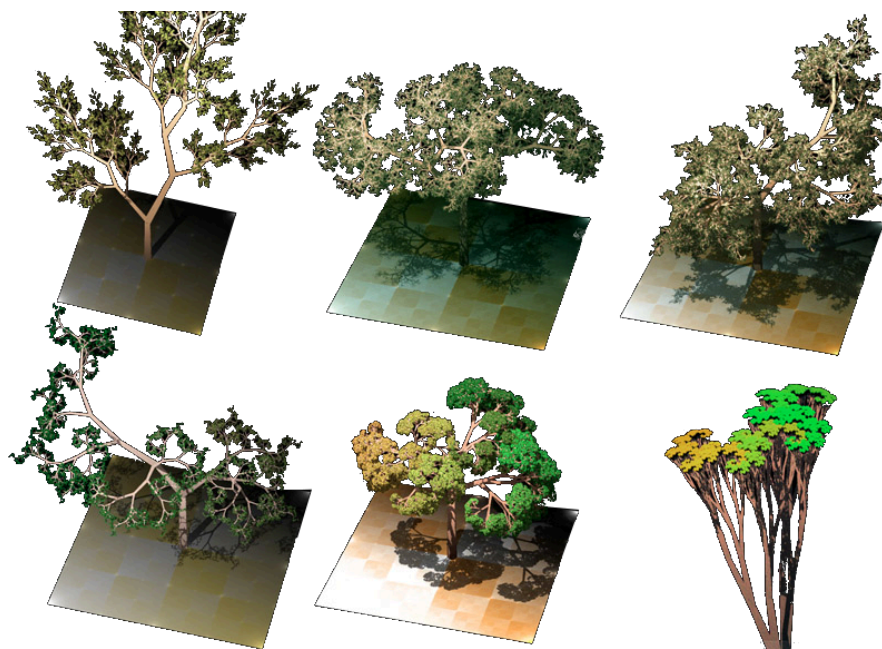


Abbildung 3: Mit L-Systems generierte Bäume (Solkoll)

2.3.2 Graph-based Model Synthesis

Graph-based Model Synthesis (Merrell, „Example-Based Procedural Modeling Using Graph Grammars“) erweitert das Konzept von Graph-Grammatiken, indem Regeln automatisiert aus einem Beispiel abgeleitet werden. Das heißt, dass ein bestehendes Modell in kleine Strukturelemente zerlegt wird. Daraus werden Regeln abgeleitet, wie Teilgraphen ersetzt werden dürfen. Durch wiederholtes Anwenden dieser Regeln entstehen neue Modelle, die lokal den gleichen Regeln des Beispiels folgen, aber global andere Strukturen haben können. Im Vergleich zu L-Systems können Regeln hier auf beliebige Graphstrukturen verweisen und komplexere räumliche Bedingungen beschreiben. Außerdem ist das Verfahren nicht auf lineare Symbolsequenzen beschränkt, wodurch es sich besser für dreidimensionale Strukturen eignet.

2.3.3 Minimale Neuberechnung

Das zentrale Problem bei L-Systems ist, dass jede Iteration unmittelbar auf der vorherigen aufbaut. Eine Regeländerung macht damit alle folgenden Iterationen ungültig, unabhängig davon, wie klein die Änderung. Bei Graph-based Model Synthesis beschreiben die Regeln lokale Nachbarschaftsbedingungen. Daher lässt sich ohne vollständige Neuprüfung nicht bestimmen, welche Teile eines generierten Modells nach einer Regeländerung noch gültig sind. In beiden Fällen fehlt eine explizite Darstellung, welche Teile des Ergebnisses von welchen Regeln abhängen.

2.4 KI-basierte prozedurale Generation

Neuronale Netzwerke können genutzt werden, um aus einem Trainingsdatensatz, der eine Menge an Beispielen enthält, ein weiteres ähnliches Beispiel zu erzeugen. In Arbeiten wie (Volz u. a.), (Bazzaz und Cooper) und (Withington und Tokarchuk) werden neuronale Modelle genutzt, um Levels aus bekannten 2D-Spielen zu reproduzieren. Dabei wird in (Volz u. a.) ein Level-Generations-Modell gegen ein Spieler-Agent-Modell, welches die „Spielbarkeit“ eines Levels bewertet, trainiert. Dieser Ansatz ist in den meisten Fällen in der Lage, spielbare Levels zu generieren. Vereinzelt werden jedoch Generationsartefakte, zum Beispiel unerreichbare Plattformen oder auch komplett leere Bereiche, erzeugt. Dazu muss gesagt werden, dass die oben genannten Veröffentlichungen „nur“ die simplen und zahlreich vorhandenen 2D-Welten des Spiels „Super Mario“ als Trainingsdatensätze heranziehen. Offen ist, ob der vorgestellte Ansatz auch in neueren und komplexeren Spielen funktioniert.

2.4.1 Terrain Diffusion

Im Ansatz „Terrain Diffusion“ werden Neuronale-Diffusion-Modelle verwendet. Diffusions-Modelle erzeugen Daten, indem sie iterativ aus zufälligem Rauschen eine strukturierte Lösung rekonstruieren. Dieses Verfahren wurde ursprünglich für die Bildgenerierung entwickelt. Heute wird es zum Beispiel auch für das Erzeugen von Höhenkarten genutzt. Hier wird ein neuronales Modell mit realen Geländedaten trainiert, damit es realistische Terrainstrukturen erzeugen kann. Um auch weiterhin unendliche Welten generieren zu können, wird das Gelände in Form von Kacheln erzeugt, die deterministisch aus einem

sogenannten „Seed“ generiert werden. So können, ähnlich wie bei klassischen Noise-basierten Verfahren, bei Bedarf neue Bereiche der Welt errechnet werden.

Der Vorteil dieses Ansatzes liegt darin, dass größere geografische Strukturen realistischer modelliert werden können als mit klassischen Noise-Funktionen.

Ein Nachteil ist jedoch, dass das Verhalten des Systems stark vom Trainingsdatensatz abhängt und Generationsregeln nicht explizit definiert sind. Dadurch ist es schwierig sicherzustellen, dass bestimmte strukturelle Anforderungen immer erfüllt werden (Goslin).

2.4.2 Minimale Neuberechnung

Bei neuronalen Modellen lässt sich nicht nachvollziehen, welche Eingaben oder Gewichte zu einem bestimmten Ausgabewert führen. Ändert sich ein Parameter, gibt es keine Möglichkeit daraus abzuleiten, welche Teile einer generierten Welt noch gültig sind. Deswegen ist ein Ziel dieser Arbeit, ein Generationssystem zu entwickeln, in dem die Regeln in jedem Fall eingehalten werden. Dies kann man mit neuronalen Modellen, die wie eine Blackbox fungieren nicht grundsätzlich sicherstellen. Deshalb wurden KI-basierte Ansätze für diese Arbeit nicht weiter betrachtet.

2.5 Houdini & Blender Geometry Nodes

Programme wie Houdini und Blender Geometry Nodes nutzen Abhängigkeits-Graph-Systeme, wie sie in dieser Arbeit vorgestellt werden. Dabei werden auch Zwischenergebnisse gespeichert und wiederverwendet, um die Laufzeit zu verkürzen. Beide Systeme sind jedoch auf die interaktive Erstellung einzelner Assets ausgelegt und nicht auf die prozedurale Generierung großer zusammenhängender Welten mit Tausenden von Elementen. Zudem sind sie geschlossene Programme, die nicht darauf ausgelegt sind, in eine Spiel- oder Simulations-Engine eingebettet zu werden. Das in dieser Arbeit vorgestellte System ist aber genau dafür konzipiert. Der Generationsalgorithmus soll in der Umgebung iterativ weiterentwickelt werden können und Änderungen in einer bereits bestehenden Welt sollen unmittelbar sichtbar werden (Side Effects Software Inc.; Blender Foundation und Blender Institute).

3 Theoretische Grundlagen

Dieses Kapitel beschreibt die technischen Konzepte, auf denen das in dieser Arbeit vorgestellte System basiert. Lazy Computation aus der funktionalen Programmierung liefert das Grundkonzept der minimalen Neuberechnung. Die Datenstrukturen in dieser Arbeit basieren auf mathematischen Graphen. Daher wird hier die genutzte Notationen erklärt. Constructive Solid Geometry wird zur Darstellung der generierten Geometrie genutzt und Grafische Programmierung bildet schließlich die Grundlage für den Editor.

3.1 Minimales Berechnen (Lazy computation)

Minimales Berechnen (englisch: lazy computation) beschreibt die Idee, in Computerprogrammen Daten erst dann zu berechnen, wenn sie benötigt werden.

Es reduziert Lag-Spikes, insbesondere beim Starten eines neuen Prozesses, da anstatt alle nutzbaren Daten nur jene berechnet werden, die aktuell benötigt werden.

In funktionalen Programmiersprachen wie Haskell findet dieses Konzept viel Anwendung und erlaubt die Nutzung unendlicher Datenstrukturen.

3.2 Graphen

Ein Graph $G := (V, E)$ besteht aus einer Menge von Knoten V und einer Menge von Kanten E . Die Funktionen $V(G) = V$ und $E(G) = E$ werden als verkürzte Notationen verwendet.

Ein einzelner Knoten wird als $v \in V$ und eine einzelne Kante als $e \in E$ geschrieben.

Eine Kante ist ein Tupel zweier Knoten $e := (v_a, v_b)$. In dieser Arbeit sind Kanten grundsätzlich von v_a nach v_b gerichtet.

Die Menge aller eingehenden bzw. ausgehenden Kanten eines Knoten v wird als $E_G^-(v)$ bzw. $E_G^+(v)$ geschrieben.

Die Menge aller Knoten, die eine eingehende bzw. ausgehende Kante zu einem Knoten haben, wird als $N_G^-(v)$ und $N_G^+(v)$ beschrieben, da sie als die Nachbarn dieses Knotens verstanden werden. In Directed Acyclic Graphs (DAG), und damit hier auch in Bäumen, werden ausgehende Nachbarn $N_G^+(v)$ auch Kinder eines Knotens genannt.

Die Notationen basieren auf den Notationen in „Modern Graph Theory“ von Béla Bollobás.

3.3 Constructive Solid Geometry

Constructive Solid Geometry (CSG) ist eine Methode zur Darstellung von Volumen, bei der komplexe Geometrien durch Kombination primitiver Geometrien (Box, Kugel etc.) dargestellt werden.

Diese Kombinationen umfassen standardmäßig Vereinigung, Schnittmenge und Differenz, wurden jedoch in späteren Arbeiten um komplexere Operationen wie Verformung und Duplikation erweitert.

CSG werden als Directed Acyclic Graph (DAG) gespeichert. Jeder Knoten ist eine implizite Operation auf den Volumen der Kindknoten. Die Blätter sind hingegen durch Parameter definierte primitive Geometrien.

Der Vorteil dieser Darstellung ist, dass die Geometrie über Parameter und Operationen beschrieben wird. Änderungen von Positionen, Größen oder anderen Parametern wirken sich unmittelbar auf das resultierende Volumen aus, ohne dass die gesamte Geometrie neu definiert werden muss.

Jedoch eignen sich CSG selten zum direkten Rendering mit Raytracing, da die Operationen zu leistungsaufwendig sind, um mit jedem Ray den DAG rekursiv zu iterieren. Zudem wächst die Größe des DAGs schnell mit der Komplexität der CSG-Repräsentation (Requicha und Voelcker; Wassermann u. a.).

3.4 Grafische-Programmierung

Grafische Programmierung ist eine Form der Programmierung, bei der Logik nicht als Code definiert ist, sondern als Diagramm in einem grafischen Editor. Funktionen werden als Knoten mit Input und Output dargestellt. Diese können mit Linien verbunden werden, um logische Abfolgen zu definieren.

Grafische Programmierung gibt einen besseren Überblick über Programmabschnitte und ist durch ihren intuitiven Syntax leichter für Laien zu verstehen.

Grafische Programme werden ausgeführt, indem die Operationen als Graph dargestellt werden. Dieser kann, vergleichbar mit funktionalen Programmiersprachen, rekursiv gelöst werden.

Jedoch ist es auch möglich, grafische Programme in Code zu übersetzen und zu Maschinencode zu kompilieren. Dies kann große Leistungsvorteile bringen.

Populäre Programme, in denen grafische Programmierung verwendet wird, sind Unity Shader Graphs, Unreal Engine Templates und Blender Geometry Nodes.

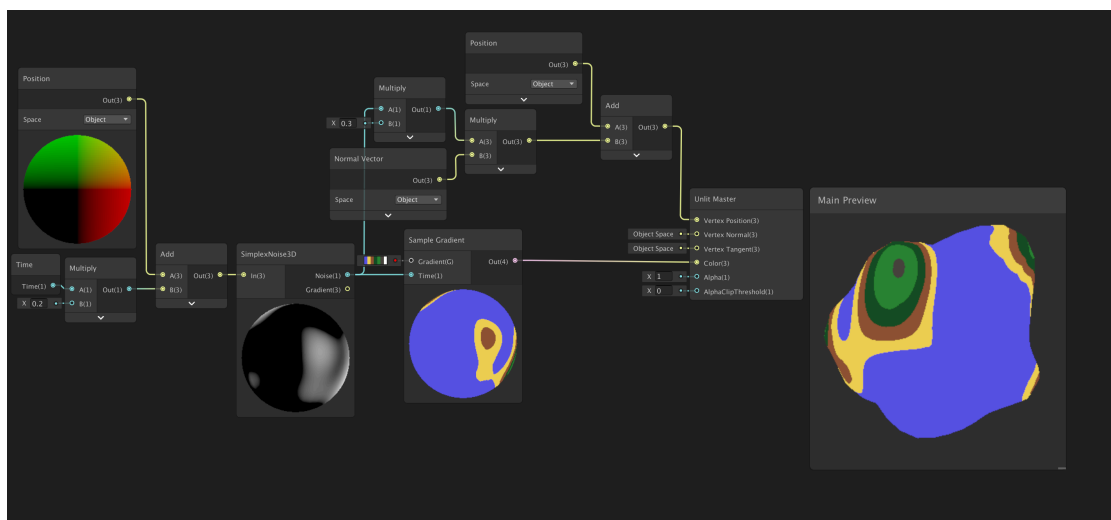


Abbildung 4: Der grafische Editor von Unity Shader Graph (Coster)

3.5 Violin Plot

Ein Violin Plot ist eine Methode zur Visualisierung der Verteilung von Messwerten. Er kombiniert die Eigenschaften eines Box Plots mit einer Kerneldichte-Schätzung.

Die Kerneldichte-Schätzung ermittelt aus einer endlichen Stichprobe eine kontinuierliche Wahrscheinlichkeitsdichtefunktion. Dafür wird um jeden Messpunkt eine Kernelfunktion gelegt. Häufig wird hierfür eine Gaußkurve verwendet. Anschließend werden alle Kernelfunktionen aufsummiert. Die Bandbreite der Kernelfunktion bestimmt, wie glatt die resultierende Dichte ist. Sie wird mit Scott's Rule automatisch aus der Stichprobengröße und Streuung berechnet (Scott).

In der Darstellung wird die geschätzte Dichte symmetrisch um eine vertikale Achse gespiegelt. So entsteht eine charakteristische geschwungene Form. Die Breite des Violins an einer bestimmten Stelle entspricht der geschätzten Häufigkeit von Messwerten in diesem Bereich. Ein weißer Punkt in der Mitte markiert den Median der Messreihe.

Im Vergleich zu einem Box Plot zeigt ein Violin Plot nicht nur Lage und Streuung, sondern auch die Form der Verteilung. So ist zum Beispiel erkennbar, ob mehrere Häufungspunkte vorliegen oder die Verteilung stark asymmetrisch ist. Dies ist bei Laufzeitmessungen besonders relevant, da diese oft durch gelegentliche Ausreißer nach oben verzerrt sind.

4 Mein Lösungsansatz

Dieses Kapitel beschreibt, wie das Ziel, die Neuberechnungszeit zu verkürzen, erreicht werden kann. Zunächst erkläre ich, warum ich ein System basierend auf einem Abhängigkeits-Graph gewählt habe. Dann werden die Komponenten des Systems vorgestellt und zentrale Algorithmen werden erklärt. Weiterhin werden Besonderheiten des Systems sowie mögliche Probleme diskutiert.

Mein Lösungsansatz beschreibt, wie prozedurale Generierung strukturiert sein sollte, damit das System automatisch erkennt, welche Teile der Welt es bei einer Änderung im Algorithmus neu berechnen muss. Dafür wird der Algorithmus als Abhängigkeits-Graph modelliert.

4.1 Minimale Neuberechnung mit Model Synthesis

Die initiale Idee meiner Arbeit entstand durch die Fragestellung, ob ein mit Model Synthesis generiertes Ergebnis nach Regeländerung minimal Neuberechnen kann. Ich habe diesen Ansatz verworfen, da man beim Versuch, Model Synthesis Ereignisse an geänderte Regeln anzupassen, einen entscheidenden Vorteil von Model Synthesis verliert:

Angenommen ein vollständiges Ereignis wurde mit dem Model-Synthesis-Algorithmus generiert. Dies ist ein Gitter an Feldern. Jedem Feld ist ein Wert zugeordnet. Dazu gibt es eine Liste an validen Nachbarkombinationen.

Wenn sich anschließend die zugrunde liegenden Regeln ändern, kann man grundsätzlich nicht sicherstellen, dass das Ereignis den neuen Regeln entspricht.

Um das Ergebnis an die neuen Regeln anzupassen, müssten alle Felder identifiziert werden, deren aktuelle Werte gegen die neuen Regeln verstoßen. Für diese Felder müssten anschließend wieder mehrere mögliche Werte zugelassen werden, sodass der Model-Synthesis-Algorithmus erneut eine konsistente Konfiguration finden kann.

Dabei zeigt sich jedoch ein grundlegendes Problem: Wenn einem Feld neue mögliche Werte hinzugefügt werden, sind diese zunächst nicht notwendigerweise mit den aktuellen Werten der Nachbarn kompatibel. Damit die lokalen Regeln wieder erfüllt sind, müssten auch den Nachbarn zusätzliche mögliche Werte hinzugefügt werden. Dieser Prozess kann sich wiederum auf deren Nachbarn ausbreiten und so weiter.

Würde man dieses Verfahren naiv implementieren, indem für alle betroffenen Nachbarn wieder alle möglichen Werte zugelassen werden, entstünde im Extremfall erneut ein vollständig unentschiedenes Gitter. In diesem Fall würde der Algorithmus faktisch wieder bei einem normalen Model-Synthesis-Prozess beginnen, wodurch kein Vorteil gegenüber einer vollständigen Neugenerierung entstünde.

Um dennoch einen Nutzen aus diesem Ansatz zu ziehen, müsste man eine Menge an Werten für Felder finden, die hinzugefügt werden sollen, sodass anschließend wieder eine konsistente Konfiguration existiert. Diese Menge sollte idealerweise minimal sein, damit möglichst große Teile der bestehenden Welt unverändert bleiben können. Gleichzeitig

müsste der Aufwand zur Bestimmung dieser Menge deutlich geringer sein als eine komplette Neugenerierung der Welt.

Eine theoretische Möglichkeit bestünde darin, den Raum der möglichen Werteerweiterungen systematisch zu durchsuchen. Beispielsweise könnte eine Breitensuche über den Graphen der möglichen Wertekombinationen durchgeführt werden, um eine minimale Menge an Änderungen zu finden, die wieder zu einer konsistenten Konfiguration führt. Allerdings wächst dieser Suchraum sehr schnell und führt in Bezug auf Laufzeit als auch Speicherverbrauch zu erheblichen Komplexitätsproblemen.

Der Vorteil des ursprünglichen Model-Synthesis-Algorithmus liegt darin, dass zu jedem Zeitpunkt alle noch möglichen Kombinationen eine valide Lösung darstellen. Das Finden einer minimalen Erweiterung dieser Mengen, die nach einer Regeländerung wieder eine gültige Lösung ermöglicht, ist jedoch wesentlich komplexer als die ursprüngliche Generierung selbst.

Daher habe ich für meine Arbeit ein Ansatz basierend auf einem Abhängigkeits-Graphen verfolgt.

4.2 Abhängigkeits-Graph

Jede mathematische Formel oder jeder Algorithmus kann als Graph an Abhängigkeiten dargestellt werden. Dabei beschreibt jeder Knoten eine mathematische Operation, die aus einem oder mehreren Eingangswerten ein Ergebnis errechnet. Dabei gilt, dass als Eingangswerte für Knoten die Ergebnisse anderer Knoten im Graphen verwendet werden. Dies bezeichne ich als Abhängigkeit, weil für die Berechnung eines Knotens alle Ergebnisse, die hierfür Eingangswerte sind, zuvor errechnet werden müssen. Das Ergebnis eines Knotens hängt nur von seinen Eingangswerten ab und hat damit keine Nebeneffekte (siehe Funktionale Programmierung in Abschnitt 3.1). Knoten und deren Ergebnisse, die keine Eingangswerte haben, bezeichne ich als konstant.

4.3 Framework

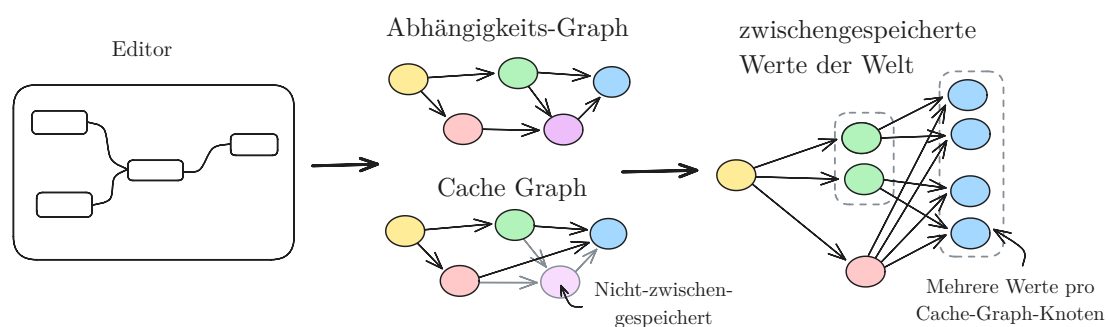


Abbildung 5: Überblick über Editor, Abhängigkeits-Graph, dessen Cache und der Welt
Mein Generationssystem besteht aus drei Bestandteilen:

1. Der grafische Editor, mit dem ein Nutzer einen Abhängigkeits-Graphen erstellen und bearbeiten kann

2. Das Template ist eine Datenstruktur, die vom Editor erstellt wird. Es enthält den Abhängigkeits-Graph sowie Anleitungen, wie dieser generiert und zwischengespeichert werden soll (Cache-Graph).
3. Der Generator vergleicht das aktuelle Template mit dem neuen Template und generiert jene Bestandteile der Welt neu, die nicht dem neuen Template entsprechen.

4.4 Grafischer Editor

Zur interaktiven Definition des Abhängigkeits-Graphen wird ein grafischer Programmierungseeditor, vergleichbar mit Unreal Templates, Blender Geometry Nodes oder Unity Shader Graph, genutzt.

Der Nutzer kann Knoten erstellen, die einer Operation entsprechen, und diese auf einer unendlichen Fläche frei anordnen. Diese Operationen haben auf ihrer linken Seite eine Liste mit Eingangswerten und auf ihrer rechten Seite eine Liste mit Ergebnissen.

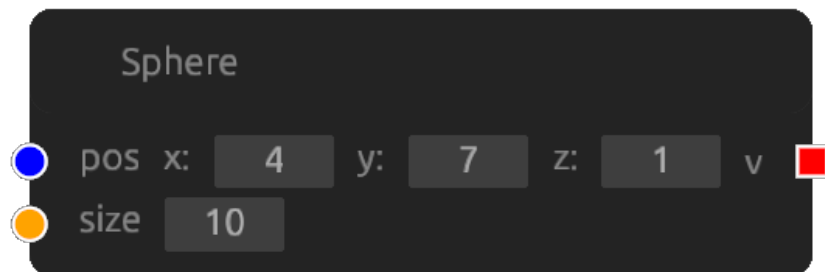


Abbildung 6: Node um ein Kugel Volumen zu definieren

Die Eingangswerte und Ergebnisse sind je nach Datentyp farbig kodiert und können mit Linien verbunden werden. Dies zeigt, dass ein Ergebnis als Eingangswert für eine andere Operation verwendet werden soll.

Um komplexe Algorithmen zu modellieren, werden Knotenverbindungen von links nach rechts aufgereiht. Parallele Stränge werden übereinander angeordnet. Damit werden auch komplexe Abhängigkeiten übersichtlich dargestellt.

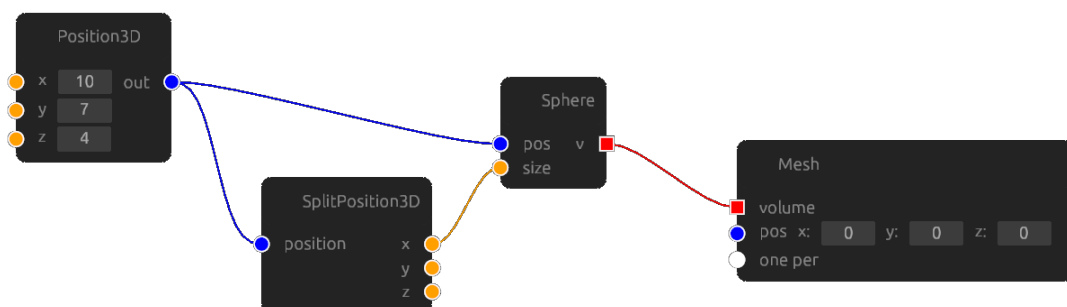


Abbildung 7: Eine Kugel, dessen Größe ihrer X Position entspricht

4.5 Template

Das Template $:= (G_{ab}, G_{ch})$ besteht aus dem Abhängigkeits-Graphen G_{ab} und einem Cache-Graphen G_{ch} .

G_{ab} ist ein Graph, der die zu generierende Welt als rekursive Formel beschreibt.

Die eingehenden Nachbarn $N_{G_{ab}}^-$ errechnen die Eingangswerte für eine Operation und die ausgehenden Nachbarn $N_{G_{ab}}^+$ sind alle Operationen, die das Ergebnis benötigen.

G_{ch} enthält einen Knoten für jeden Knoten in G_{ab} , der zwischengespeichert werden soll. Dies ist eine Untermenge aller Knoten in G_{ab} $V(G_{ch}) \subset V(G_{ab})$.

Bei der Entscheidung, wie groß diese Untermenge sein soll, müssen der „Overhead“ durch Zwischenspeicherung und die Zeitersparnis durch Wiederverwendung der Ergebnisse abgewogen werden.

Die eingehenden Nachbarn $N_{G_{ch}}^-$ sind alle Caches, von denen der Knoten abhängt. Man findet diese, indem man den Baum der Abhängigkeiten in G_{ab} in allen seinen Verzweigungen rekursiv durchsucht, bis man jeweils auf einen Knoten in G_{ch} stößt. Zur Veranschaulichung betrachte Abbildung 5 und Abbildung 8.

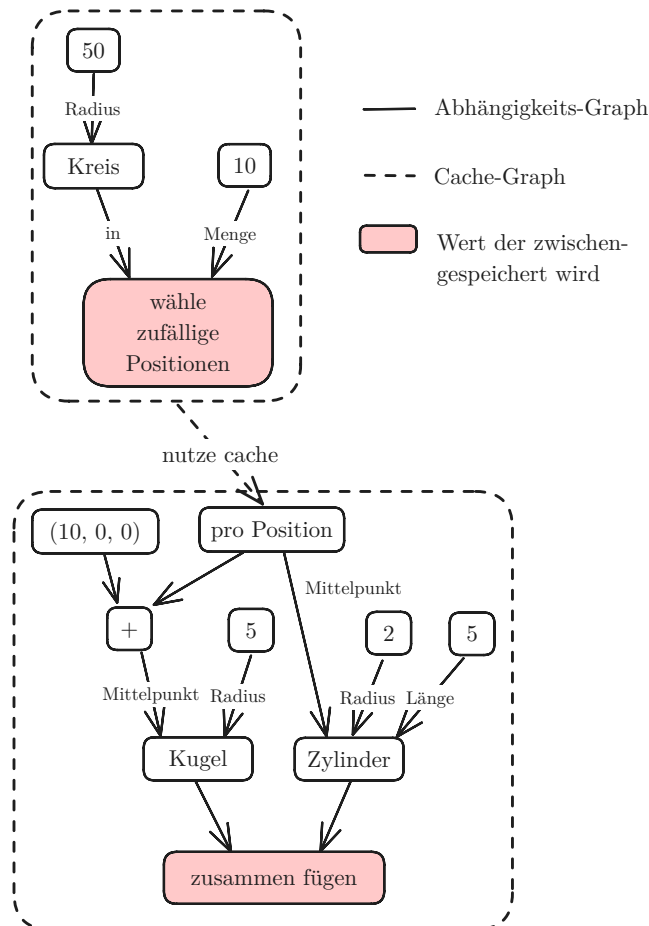


Abbildung 8: Beispiel eines Abhängigkeites-Graphen und dessen Cache-Knoten zur Generierung einer Menge an Bäumen

4.5.1 Level

Wenn der G_{ab} ein Directed Acyclic Graph (DAG) ist, kann jedem Knoten $v \in G_{ab}$ ein Level $l(v)$ zugeordnet werden. Dies ist definiert als:

$$l(v) > l(v_i) \quad \forall v_i \in N_{G_{ab}}^-(v)$$

Somit ist das Level eines Knotens immer größer als das Level aller Knoten, von denen der Knoten abhängt.

Um den G_{ab} zu errechnen, werden die Knoten der Level aufsteigend errechnet. Innerhalb eines Levels hat die Reihenfolge keine Auswirkung.

4.5.2 Einen prozeduralen Algorithmus als Template darstellen

Für die Implementierung habe ich mich entschieden, dass das finale Ergebnis des Templates ein Volumen als Constructive Solid Geometry (CSG) sein soll.

Diese CSG setzt sich aus Remove- und Union-Operationen auf primitiven geometrischen Körper wie Kugeln und Boxen zusammen.

Einige der Operationen, die ich implementiert habe, sind:

- eine Kugel aus Position und Durchmesser definiert
- eine Box aus Position und Seitenlänge definiert
- alle Positionen auf einem Gitter, die innerhalb eines Volumens sind, errechnen
- eine Menge an zufälligen Positionen innerhalb eines Volumens errechnen
- Addition, Subtraktion, Multiplikation und Division von Positionen und Zahlen

4.5.3 Generation eines Templates

Die Operationen, ein Gitter oder zufällige Positionen zu berechnen, erzeugen eine Menge an Werten (mehrere Positionen). Operationen, die diese Mengen verwenden, können entweder die Operationen auf die gesamte Menge anwenden, zum Beispiel eine Filter-Operation, oder wenden die Operationen auf jedes einzelne Element separat an, zum Beispiel „Platziere an jede Position ein Baum.“

Die Fähigkeit, weitere Operationen pro Element auszuführen, ermöglicht es, iterativ immer feiner werdende Details zu generieren.

In meinem System arbeiten alle Algorithmen nur auf den Knoten des Templates. Denn Algorithmen auf dem Template haben einen klaren Laufzeitunterschied gegenüber Algorithmen auf der generierten Welt. Die Menge an Knoten im Abhängigkeits-Graphen und so auch im Cache-Graphen skaliert mit der Menge an Operationen des Generationsalgorithmus. Wobei die Menge der Elemente in der generierten Welt mit den Größen der Mengen an errechneten Werten skaliert. Mit anderen Worten: Alle Knoten im Template zu iterieren ist relativ schnell möglich. Hingegen kann die Laufzeit exponentiell ansteigen, wenn alle Elemente in der Welt iteriert werden.

Daher werden die Abhängigkeiten im Template verwendet, um herauszufinden, wie die Welt neu generiert werden muss.

4.6 Generator

Der Generator enthält einen Graphen G_{gen} , der dem Cache-Graphen G_{ch} im Template entspricht. Jedoch wo G_{ch} nur einen Knoten pro Operation enthält, enthält G_{gen} einen Knoten pro Ergebnis dieser Operation. Zur Veranschaulichung betrachte Abbildung 5. Jeder Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ speichert, welchem Knoten $v_{\text{ch}} \in V(G_{\text{ch}})$ er entspricht $v_{\text{ch}} = \mathbf{cache}(v_{\text{gen}})$. Dazu hat ein Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ das gleiche Level wie sein Cache-Template-Knoten $l(v_{\text{gen}}) = l(\mathbf{cache}(v_{\text{gen}}))$.

Zusätzlich enthält der Generator $:= (G_{\text{gen}}, Q_{\text{tasks}})$ eine Queue Q_{tasks} , die zwei Arten von Aufträgen auf G_{gen} nach ihren Levels sortiert.

$$\text{pop}(Q_{\text{tasks}}) := \min_{q \in Q_{\text{tasks}}} (l(q))$$

Berechnungsaufträge ermitteln das Ergebnis eines Knotens in G_{gen} . Kind-Update-Aufträge erzeugen oder löschen Kinder, bis ihre Anzahl dem Template entspricht.

4.6.1 Abhängigkeiten-Werte im Generator-Graph finden

Um einen Knoten in G_{gen} zu errechnen, benötigt man die Ergebnisse der Knoten in G_{gen} , von denen dieser Knoten abhängt. Wie in Abschnitt 4.5.3 erläutert, ist es innerhalb einer vertretbaren Laufzeit nicht möglich, diese zum Beispiel mit einer Tiefensuche zu finden.

Stattdessen wird für jeden Cache-Knoten $v \in V(G_{\text{ch}})$ einer der Knoten, von denen dieser abhängt, $N_{G_{\text{ch}}}^-(v)$ als Erstellungsknoten $v_c \in V(G_{\text{ch}})$ im Template markiert $v_c = \mathbf{create}(v)$.

Um nun für einen Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ alle weiteren Knoten zu finden, von denen dieser abhängt $N_{G_{\text{gen}}}^-(v_{\text{gen}})$, werden die relativen Schritte in G_{ch} ausgehend vom Erstellungsknoten v_c hin zu den weiteren abhängigen Knoten als Baum gespeichert $T_{\text{rel}}(v_{\text{gen}})$. Ein relativer Schritt v_{step} gibt an, dass man beginnend bei einem Knoten $v \in V(G_{\text{ch}})$ entweder aufwärts ($\mathbf{up}(v_{\text{step}}) = \text{True}$) in einen Knoten $v_{\text{up}} \in V(G_{\text{ch}})$ gehen soll, von dem v abhängt ($v_{\text{up}} \in N_{G_{\text{ch}}}^-(v)$), oder abwärts ($\mathbf{up}(v_{\text{step}}) = \text{False}$) in einen Knoten $v_{\text{down}} \in V(G_{\text{ch}})$, der von v abhängt ($v_{\text{down}} \in N_{G_{\text{ch}}}^+(v)$).

Da ein Knoten $v \in V(G_{\text{ch}})$ mehr als einen eingehenden oder ausgehenden Nachbarn haben kann, speichert ein relativer Schritt auch, in welchen Nachbarn gegangen werden soll ($\mathbf{cache}(v_{\text{step}})$). Ein relativer Schritt Speicher weiterhin, ob dieser Nachbar eine Abhängigkeit für v_{gen} ist ($\mathbf{deps}(v_{\text{step}}) = \text{True}$).

Diese relativen Schritte verwenden nur Knoten, die ein kleineres Level als v_{gen} haben. Da im Generator die Knoten im Level in aufsteigender Reihenfolge erstellt werden, ist so sichergestellt, dass alle relativen Wege existieren.

Für einen Knoten im Template kann es mehrere Knoten im Generator geben. Daher können dort pro Abhängigkeit eines Cache-Knotens auch mehrere Knoten gefunden werden.

Abbildung 9 veranschaulicht, wie Algorithmus 1 die abhängigen Werte im Generator mit Hilfe der relativen Schritte auf dem Template sucht.

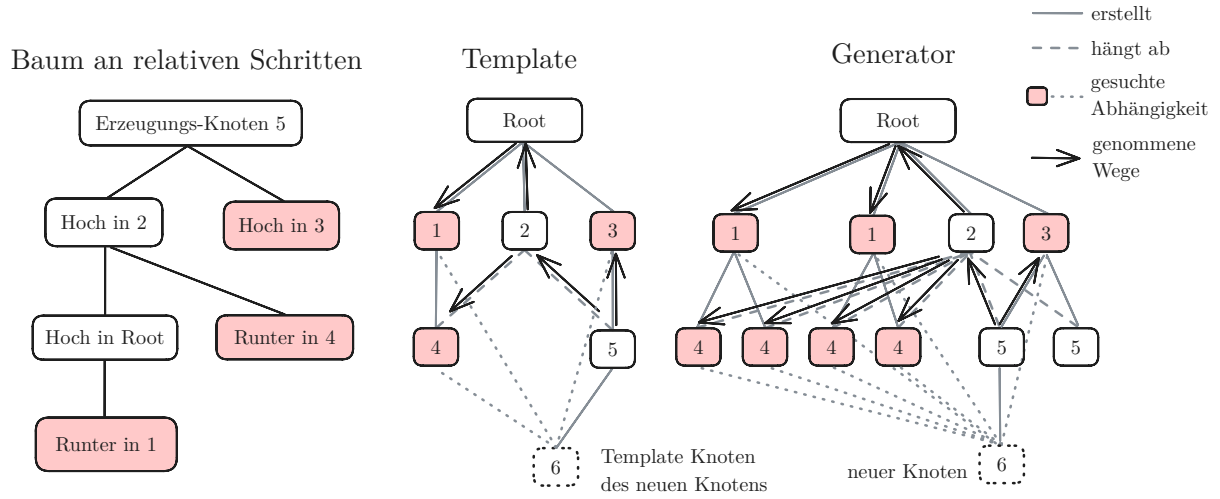


Abbildung 9: Beispiel der Anwendung eines Baums an relativen Schritten auf das Template und den Generator

Algorithmus 1: Finde abhängige Knoten in G_{gen}

```

1: procedure FINDDEPS( $v_{\text{gen}}, v_{\text{gen creates}}$ )
2:    $T \leftarrow T_{\text{rel}}(v_{\text{gen}})$ 
3:    $D \leftarrow \emptyset$ 
4:    $Q \leftarrow \emptyset$ 
5:   push( $Q, (v_{\text{root}} \in T, v_{\text{gen creates}})$ )
6:
7:   while  $Q \neq \emptyset$  do
8:      $(v_{\text{step}}, v_{\text{gen}}) \leftarrow \text{pop}(Q)$ 
9:
10:    if  $\text{deps}(v_{\text{step}})$  then
11:      push( $D, v_{\text{gen}}$ )
12:    end
13:
14:    for  $v_{\text{child step}} \in N_T^+(v_{\text{step}})$  do
15:       $N \leftarrow N_{G_{\text{gen}}}^-(v_{\text{gen}})$  if  $\text{up}(v_{\text{child step}})$  else  $N_{G_{\text{gen}}}^+(v_{\text{gen}})$ 
16:
17:      for  $v \in N$  do
18:        if  $\text{cache}(v_{\text{child step}}) = \text{cache}(v)$  then
19:          push( $Q, (v_{\text{child step}}, v)$ )
20:        end
21:      end
22:    end
23:  end
24:  return  $D$ 
25: end

```

4.6.2 Kind-Update-Aufträge

Kind-Update-Aufträge enthalten den Index des Erstellungsknoten und den Index eines Erstellungseintrags $E_{\text{create}}(v_{\text{ch}})$ in dessen Template-Knoten.

Algorithmus 2: Kinder updaten

```

1: procedure UPDATECHILD( $v_{\text{gen}}, v_{\text{ch child}}$ )
2:
3:    $C \leftarrow \{v \in N_{G_{\text{gen}}}^+(v_{\text{gen}}) \mid \text{cache}(v) = v_{\text{ch child}}\}$ 
4:
5:    $C_{\text{to delete}} \leftarrow \{v \in C \mid \neg \text{valid}(v, v_{\text{gen}})\}$ 
6:
7:   for  $v \in C_{\text{to delete}}$  do
8:     DELETE( $v$ )
9:   end
10:
11:    $n \leftarrow \text{num}(v_{\text{ch child}}, v_{\text{gen}})$ 
12:    $i \leftarrow 0$ 
13:   for  $i < n - |C \setminus C_{\text{to delete}}|$  do
14:     CREATE( $v_{\text{ch child}}, v_{\text{gen}}$ )
15:   end
16:
17: end
18:
19:
20: procedure CREATE( $v_{\text{ch}}, v_{\text{gen creates}}$ )
21:   push( $V(G_{\text{gen}}), v_{\text{new}}$ )
22:
23:   cache( $v_{\text{new}}$ )  $\leftarrow v_{\text{ch}}$ 
24:
25:    $D \leftarrow \text{FINDDEPS}(v_{\text{new}}, v_{\text{gen creates}})$ 
26:
27:    $N_{G_{\text{gen}}}^-(v_{\text{new}}) \leftarrow D$ 
28:
29:   for  $v \in D$  do
30:     push( $N_{G_{\text{gen}}}^+(v), v_{\text{new}}$ )
31:   end
32:
33:   pushCalculate( $Q_{\text{task}}, v_{\text{new}}$ )
34: end
35:
36: procedure DELETE( $v_{\text{gen}}$ )
37:
38:   for  $v \in N_{G_{\text{gen}}}^-(v_{\text{gen}})$  do
39:     remove( $N_{G_{\text{gen}}}^+(v), v_{\text{gen}}$ )
40:   end
41:
42:   for  $v \in N_{G_{\text{gen}}}^+(v_{\text{gen}})$  do
43:     DELETE( $v$ )
44:   end
45:
46:   remove( $V(G_{\text{gen}}), v_{\text{gen}}$ )
47: end

```

Dieser Erstellungseintrag definiert, wie viele Kinder es geben soll $\mathbf{num}(v_{\text{ch}}, v_{\text{gen creates}})$. Dies sind entweder genau n pro Erstellungsknoten oder hängen von dem Wert des Erstellungsknotens $v_{\text{gen creates}}$ ab, wie z.B. einer Positionsmenge. Dazu gibt $\mathbf{valid}(v_{\text{gen}}, v_{\text{gen creates}})$ an, ob ein Kind v_{gen} für den Erstellungsknoten $v_{\text{gen creates}}$ noch valide ist, also ob beispielsweise eine Position noch in der Menge an Positionen ist.

Daraufhin wird die vorhandene Menge an Kindern mit der gewünschten Menge verglichen. Bei Ungleichheit werden Kinderknoten gelöscht oder neue erzeugt. Wenn ein neuer Knoten erzeugt wird, werden mit dem Baum an relativen Schritten die Indizes aller abhängigen Knoten gesucht und im Knoten gespeichert. Algorithmus 2 veranschaulicht mit Pseudo-Code diesen Update-Prozess.

In `UpdateChild` ist $v_{\text{gen}} \in V(G_{\text{gen}})$ ein Knoten für den alle Kinder die überprüft werden sollen, welche dem Cache Knoten $v_{\text{ch child}}$ entsprechen.

4.6.3 Berechnungsaufträge

Berechnungsaufträge errechnen den Wert eines Knotens $v_{\text{gen}} \in V(G_{\text{gen}})$ neu. Dabei wird der entsprechende Knoten im Abhängigkeits-Graphen rekursiv errechnet.

Wenn der Algorithmus auf einen Knoten $v_{\text{ab}} \in V(G_{\text{ab}})$ stößt, der einen Cache-Knoten hat $v_{\text{ab}} \in V(G_{\text{ch}})$, werden die Werte der jeweiligen abhängigen Knoten von v_{gen} verwendet.

4.6.4 Abhängigkeitskreise

Das Template kann Abhängigkeitskreise enthalten. Um dennoch valide Lösungen errechnen zu können, muss es für jeden Knoten v_{val} einen validen Nullwert geben.

So kann der Abhängigkeits-Graph iterativ gelöst werden. Pro Kreis im Abhängigkeits-Graph wird eine Kante als durchgeschnitten markiert $N_{\text{cut}}^+(v) \subseteq N_{G_{\text{dep}}}^+(v) \quad v \in V(G_{\text{dep}})$. Der Abhängigkeits-Graph ohne die durchtrennte Kanten $N_{\text{not cut}}^+(v) := N_{G_{\text{dep}}}^+(v) \setminus N_{\text{cut}}^+(v)$ ist ein DAG (Directed Acyclic Graph). Folglich kann jedem Knoten ein Level $l(v)$ zugeordnet werden.

$$l(v) > l(v_i) \quad \forall v_i \in N_{\text{not cut}}^+(v)$$

Die Knoten werden Level für Level erzeugt. So wird sichergestellt, dass alle nicht-geschnittenen Abhängigkeiten bereits errechnet wurden, wenn der Knoten selbst errechnet wird. Hat ein Knoten geschnittene Abhängigkeiten, werden für diese im ersten Durchlauf ihr Nullwert verwendet. Jeder Knoten, der Nullwerte für seine geschnittenen Abhängigkeiten genutzt hat, wird erneut errechnet, sobald alle Knoten einmal errechnet wurden. Nun können die Ergebnisse der letzten Generation anstatt der Nullwerte verwendet werden. Dies wird so oft wiederholt, bis keine Nullwerte mehr verwendet werden.

4.7 Implementierung

Für meine Implementierung habe ich Rust als Programmiersprache gewählt, da sie erlaubt, speichersicheren Lowlevel-Code zu schreiben, um die Laufzeit von Algorithmen effektiv zu verbessern. Zudem hat sie einen (im Gegensatz zu C++) umfassenden und einfach zu nutzenden Package-Manager.

4.7.1 Stabile Listen

Alle Graphen sind mit stabilen Listen implementiert. Eine stabile Liste ist ein sich automatisch vergrößerndes Array. Wenn das aktuelle Array voll ist, wird ein größeres Array alloziert und die Werte mit einer Memcopy-Operation kopiert. Die Indizes von Elementen ändern sich in stabilen Listen nicht. Wenn ein Element entfernt wird, werden die weiteren Elemente nicht verschoben, um die Lücke zu schließen. Stattdessen wird der Index des nächsten freien Elements gespeichert. Dazu speichert die Liste den Index des ersten freien Elements, welches genutzt wird, wenn ein neues Element eingefügt wird. Der dort gespeicherte nächste freie Index wird dann als erster freier Index gespeichert. Dies erlaubt Einfüge-, Entfernungs- und Zugriffslaufzeit von $O(1)$.

Weiterhin wird pro Element eine Versionsnummer gespeichert. Bei einem Zugriff wird die Version des Indexes mit der Version des Elements an der Stelle des Index verglichen. Indizes bleiben valide, solange das Element in der Liste ist. Und es wird erkannt, wenn man mit einem veralteten Index zugreift.

Die Versionsnummer wird in den oberen 32 Bit des Indexes gespeichert. Somit können in einer stabilen List 2^{32} Elemente gespeichert werden.

Stabile Listen ermöglichen es, Graphen effizient als Listen darzustellen, indem in den Knoten die Indizes der anderen Knoten gespeichert werden, zu dem Kanten existieren. Stabile Listen sind dabei schneller als Hash Maps (Peters).

4.7.2 Multi Threading

Der Generator sowie die Sampling-Operationen werden in asynchronen Workern ausgeführt. Für die Kommunikation werden Channel verwendet (Hoare; Glavina).

Der Editor läuft im Render-Thread und errechnet bei jeder Änderung das aktuelle Template. Dieses wird mit einem Channel zum Generator gesendet. Der Generator vergleicht sein aktuelles Template mit dem neuen, berechnet alle benötigten Änderungen in der Welt und speichert das neue Template als sein aktuelles.

Die errechnete CSG-Darstellung der Welt wird mit einem weiteren Channel an die Sampler gesendet, welche die CSG-Darstellung in ein Voxel DAG oder Mesh umrechnen und auf die GPU transferieren.

4.7.3 Small Vectors

Für die Zwischenspeicherung der Werte werden Small Vectors verwendet. Diese haben die Eigenschaft, dass die ersten n Elemente direkt auf dem Stack alloziert werden. Sobald diese voll sind, wird ein Array auf dem Heap alloziert.

Alle Werte müssen als Liste behandelt werden, da ein Knoten für einen Input immer von mehreren Knoten abhängen kann. Jedoch enthält diese Liste meist nur ein Element. Small Vektoren erlauben es, für diese Fälle auf das Allokieren des Heap zu verzichten (The Servo Project Developers).

4.7.4 Output Datenstruktur

Das Kernkonzept, einen sehr großen Graphen, der einen prozeduralen Algorithmus darstellt, zu bearbeiten, indem man die Abhängigkeiten in einem vergleichbaren kleineren Template nutzt, enthält keine konkreten Annahmen über die Art der Geometrie. In meiner Implementierung habe ich CSGs genutzt, da es eine allgemeine Form ist, Volumen darzustellen, und diese zudem leicht zu bearbeiten sind.

Aber gerade CSGs mit vielen Knoten sind nicht performant zu rendern. Deshalb können sie in meiner Implementierung entweder als Sparse Voxel DAGs oder mit Marching Cubes diskretisiert werden.

Voxel-Datenstrukturen können relativ effizient mit Ray Marching gerendert werden (Amanatides und Woo; Laine und Karras). Mit Hilfe von Marching Cubes kann ein CSG als Mesh approximiert werden (Lorensen und Cline).

4.8 Beispiele

Im Folgenden werden mehrere Beispiele vorgestellt, die die Funktionsweise des Systems demonstrieren. Dabei wird zuerst in minimalen Beispielen gezeigt, wie das System Teile der generierten Welt wiederverwenden kann. Danach wird eine komplexe Welt vorgestellt, welche den Umfang des Systems veranschaulicht.

4.8.1 Nur finales Volumen neu errechnen

Im ersten Beispiel wird eine große Anzahl von Bäumen generiert, die auf einem Gitter verteilt sind. Jeder Baum besteht aus einem Stamm (Zylinder) und einer Baumkrone (Kugel). Die Positionen aller Bäume werden mit einer Gitter-Operation berechnet und im Cache gespeichert. Die Form der Baumkrone wird als separate Operation im Abhängigkeits-Graphen modelliert.

Wenn nun die Form oder die Größe der Baumkrone verändert wird, betrifft diese Änderung nur den Teilgraphen, der die Baumkrone berechnet. Dabei können die Positionen der Bäume wiederverwendet werden. Der Generator erkennt, dass nur die CSG-Volumen der Baumkronen ungültig sind und berechnet ausschließlich diese neu.

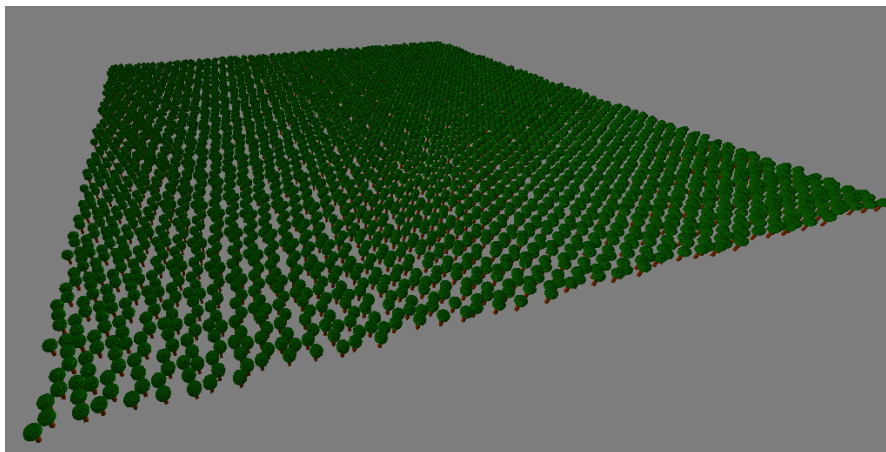


Abbildung 10: Baum-Mengen-Beispiel

4.8.2 Extern kontrollierte Variable

Knoten im Abhängigkeits-Graphen können nicht nur von anderen berechneten Knoten abhängen, sondern auch von extern kontrollierten Variablen. Ein typisches Beispiel ist die Kameraposition, die von der Game-Engine zur Verfügung gestellt wird und sich kontinuierlich verändern kann.

In diesem Beispiel wird die Kameraposition als konstanter Knoten im Abhängigkeits-Graphen modelliert. Darauf aufbauend wird eine Positionsmenge berechnet, die alle Weltabschnitte (Chunks) innerhalb eines bestimmten Radius um die Kamera enthält. Da diese Menge direkt von der Kameraposition abhängt, wird sie neu berechnet, sobald sich die Kameraposition ändert.

Bewegt sich die Kamera, erkennt der Generator, dass der Wert des Knotens für die Kameraposition ungültig geworden ist und berechnet alle abhängigen Knoten neu. Chunks, die nun außerhalb des Radius liegen, werden gelöscht. Gleichzeitig werden für

Chunks, die nun innerhalb des Radius sind, neue Knoten erzeugt und deren Werte berechnet. Chunks, die sich weiterhin im gültigen Bereich befinden, bleiben unverändert im Cache und müssen nicht neu berechnet werden.

Dieses Prinzip lässt sich über die Kameraposition hinaus auf weitere extern kontrollierte Variablen übertragen. Für ein LOD-System kann die Entfernung zur Kamera genutzt werden, um die Detailtiefe einzelner Objekte zu bestimmen. Auch für View Culling kann die Blickrichtung der Kamera als externe Variable genutzt werden, um nur sichtbare Weltbereiche zu generieren und nicht sichtbare zu verwerfen. Darüber hinaus können auch Variablen genutzt werden, die von Spielständen abhängen, um Teile der Welt dynamisch zu laden oder zu verändern, ohne den Rest der Welt neu zu errechnen. Beispiele dafür sind, ob ein bestimmtes Gebiet betreten wurde oder ein definiertes Ereignis eingetreten ist.

Extern kontrollierte Variablen ermöglichen es daher, das System nicht nur für statische Änderungen am Generationsalgorithmus einzusetzen, sondern auch für dynamische Änderungen, auf die Spieler reagieren können.

4.8.3 Höhlen-Beispiel

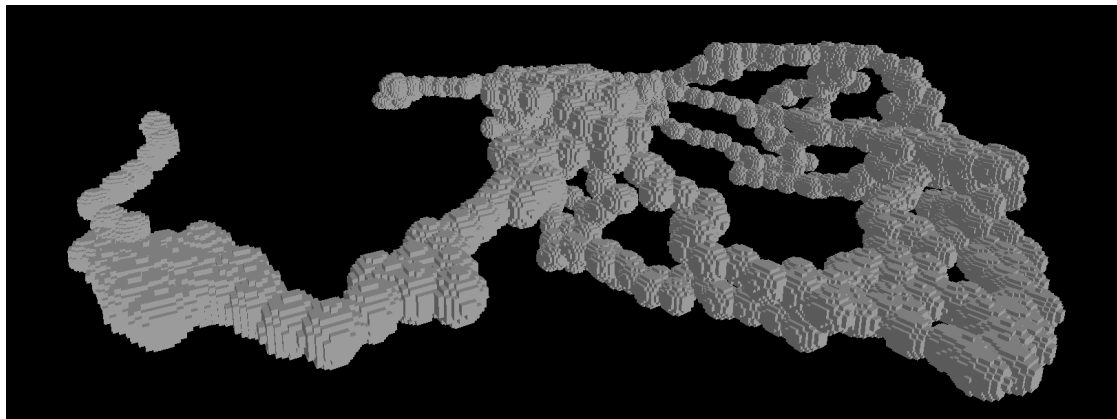


Abbildung 11: Höhlen-Beispiel

Dieses Beispiel zeigt, wie simple zufällige Höhlen-Tunnel generiert werden können. Hierbei werden mehrere Kreuzungspunkte in einem Disk-Volumen zufällig gewählt. Kreuzungspunkt-Paare werden dann mit zum Endpunkt gerichteten Random Walks verbunden. Dabei wird pro Schritt der Normalvektor zum Endpunkt mit einem Vektor addiert. Mit der Gewichtung der beiden Vektoren lässt sich die Stärke des Random Walks steuern.

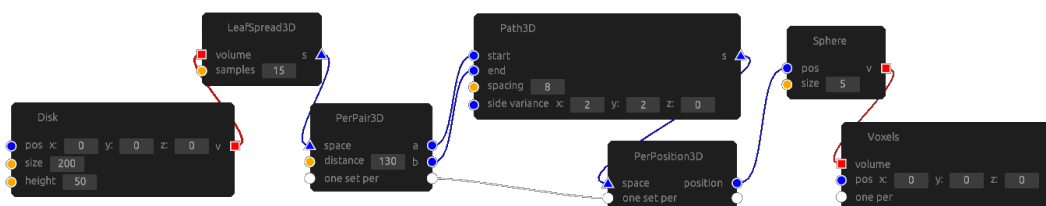


Abbildung 12: Graph für Höhlen-Beispiel

4.8.4 Insel-Beispiel

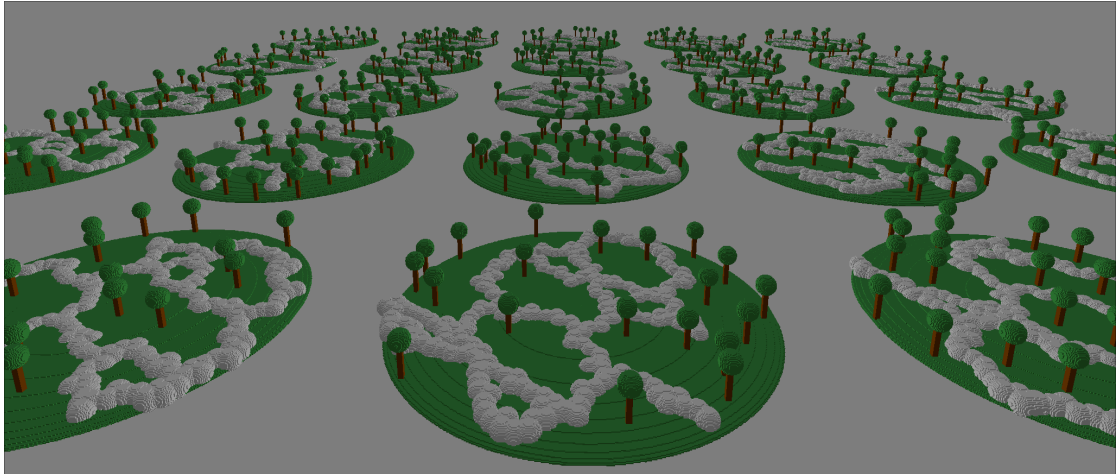


Abbildung 13: Insel-Beispiel

Dies ist ein komplexeres Beispiel, welches den Umfang meiner Implementierung darstellt. In einem rechteckigen Generationsbereich werden Inseln in regelmäßigen Abständen platziert. Auf jeder Insel wird eine Menge an zufälligen Kreuzungspunkte generiert. Kreuzungspunkte, die in der Nähe von einander sind, werden mit zufällig verlaufenden Wegen verbunden (graue Kugeln). Danach werden die ungenutzten Flächen der Insel mit zufällig platzierten Bäumen gefüllt.

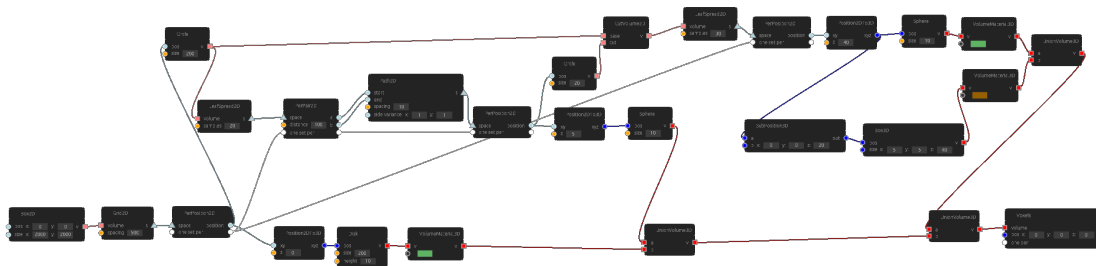


Abbildung 14: Graph für Insel-Beispiel (Hochauflösende Version im Anhang)

5 Analyse

Die Bewertung eines Systems zur prozeduralen Generierung ist nicht trivial, da unterschiedliche Systeme häufig unterschiedliche Ziele verfolgen. Während einige Ansätze primär auf maximale Generationsgeschwindigkeit oder realistische Ergebnisse optimiert sind, liegt der Fokus dieser Arbeit auf der effizienten Neuberechnung nach Änderungen am Generationsalgorithmus. Ziel ist es, bei kleinen Änderungen am Generationsprozess möglichst große Teile der bereits berechneten Welt wiederverwenden zu können.

Im nächsten Abschnitt wird die theoretische Laufzeitkomplexität des Systems analysiert.

Um den realen Zeitgewinn bei der Neuberechnung zu untersuchen, werden Benchmarks auf mehreren Beispielwelten genutzt, und es wird abgeschätzt, wieviel Overhead das System gegenüber einer minimalen Implementierung eines Generationsbeispiels hat.

Neben den reinen Laufzeiteigenschaften spielt auch die Nutzerfreundlichkeit des Systems eine Rolle. Durch den grafischen Editor lassen sich komplexe Abhängigkeiten oft leichter überblicken als in klassischem Quellcode. Jedoch erhöht dieser Ansatz auch die Komplexität des Systems. Während klassischer Programmcode sehr flexibel ist und ohne zusätzliche Struktur auskommt, erfordert der hier vorgestellte Ansatz eine explizite Modellierung aller Abhängigkeiten im Graphen.

Im Abschnitt 5.5 sollte der Aufwand für die Erweiterung des Systems betrachtet werden. Meine Implementierung nutzt nur einfache Operationen auf CSGs. Dies entspricht nicht den tatsächlichen Datenstrukturen und Problemen in Spielen oder Simulationen. Daher würde eine große Menge an weiteren Operationen und Datentypen implementiert werden müssen.

5.1 Theoretische Laufzeit

Die Kernidee des Systems ist, dass der Generationsalgorithmus der Welt als Abhängigkeits-Graph definiert ist. Annahme: Der Abhängigkeits-Graph hat a Knoten. Die errechneten Ergebnisse von manchen Knoten im Abhängigkeits-Graphen werden zwischengespeichert. Dies ist durch den Cache-Graph definiert. Dieser hat c Knoten, wobei $c \leq a$ ist. Wir definieren einen Cache-Faktor als $c_f := \frac{c}{a}$. Ein Knoten im Abhängigkeits-Graphen kann mehrere Knoten im Generator haben und jeder dieser Knoten kann mehrere Kinder Knoten für ein Kind im Abhängigkeits-Graph haben. Daher definieren wir einen Branche-Faktor b_f , der die durchschnittliche Menge an Kindern pro Abhängigkeits-Knoten definiert. Zuletzt müssen bei einer Änderung des Abhängigkeits-Graphen nur manche Knoten neu berechnet werden. Den Faktor der neu zu berechnenden Knoten im Generator nennen wir g_f .

Somit ist die Laufzeitkomplexität einer Errechnung nach Template-Änderung oder Änderung einer externen Variabel:

$$O\left((g_f c_f a)^{b_f}\right) = O(a^{b_f})$$

Dagegen ist die Laufzeit, ein Template zu berechnen, $O(a^2)$. Die quadratische Laufzeit entsteht, da pro Cache-Knoten die relativen Pfade berechnet werden müssen. Bei großen Welten mit vielen Details kann b_f wesentlich größer als 2 sein und Faktoren im Bereich von 10 – 100 sind nicht unrealistisch.

5.2 Reduktion der Neuberechnungszeit

Um die Reduktion der Neuberechnungszeit zu bewerten, wird untersucht, wie stark sich die Laufzeit im Vergleich zu einer vollständigen Neugenerierung reduziert.

Hier für habe ich mehrere Knoten in den Beispiel-Graphen ausgewählt und jeweils einen als geändert markiert. Danach wird das Graph neu werden und errechnet. Diese Knoten sind so gewählt, dass zunehmend mehr Cache Knoten nicht neu errechnet werden müssen.

5.2.1 Höhlen-Beispiel

Die Neuerrechnung nach Änderung eines Knoten wird automatisiert durch 50 Iterationen aufgewärmt und danach 500 mal zeitlich gemessen. Hierfür wird die Linux Time API genutzt. Alle Benchmarks wurden nacheinander auf der gleichen Maschine ausgeführt. Parallel liefen keine weiteren ressourcenintensiven Programme.

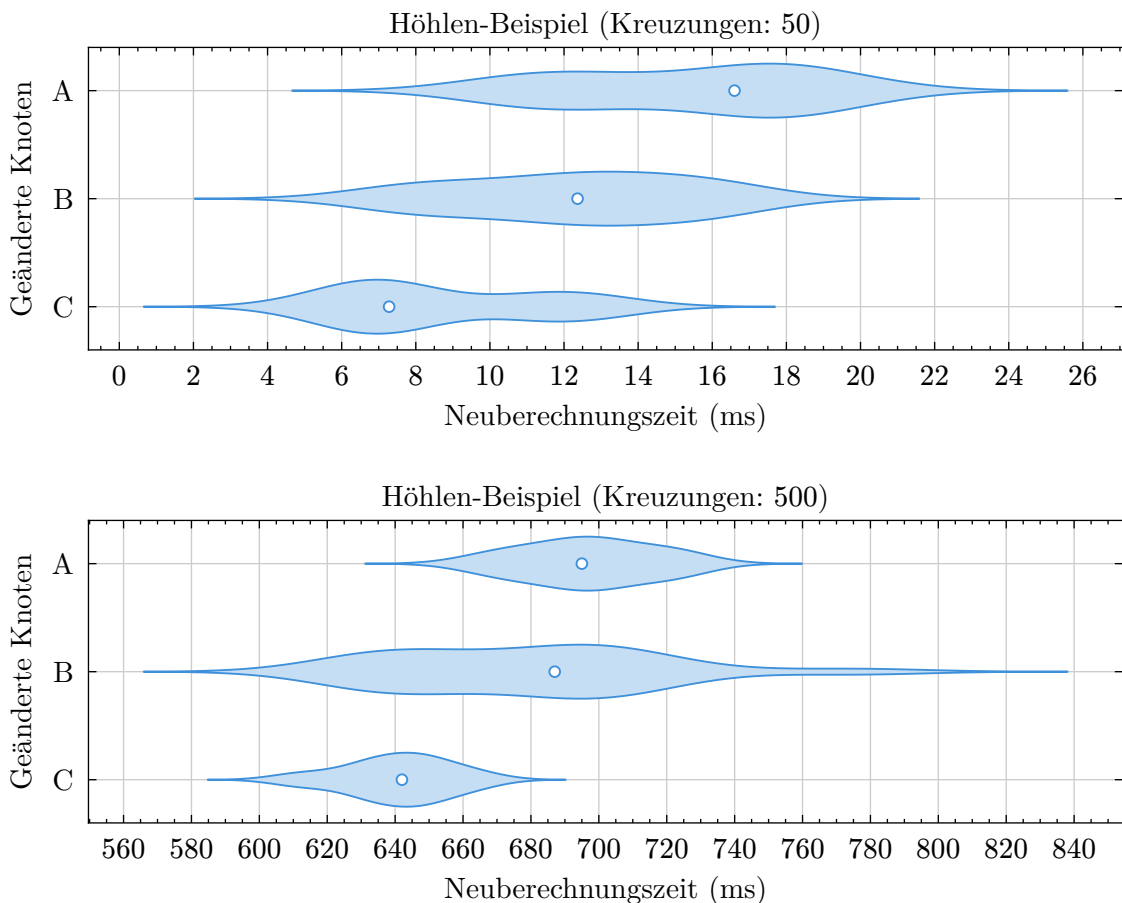


Abbildung 15: Unterschied der Neuberechnungszeit zwischen verschiedenen geänderten Knoten im Höhlen-Beispiel

Die Verteilungsdichte der Messwerte werden als Violin Plot (siehe Abschnitt 3.5) dargestellt. Der weiße Punkt ist der Median der Messwerte.

Die Bandbreite zur Kerneldichte-Bestimmung wurde mit Scott's Rule (Scott) errechnet. Für diesen Benchmark habe ich folgende Knoten im Höhlen-Graphen ausgewählt. Knoten A ist das linke Disk-Volumen, welches den Generationenbereich definiert. Wenn dieses geändert wird, muss die gesamte Welt neu errechnet werden. Für B habe ich den Wegknoten ausgewählt der, die Höhlen-Tunnel definiert. Hier werden die Positionen der Tunnel-Kreuzungen wiederverwendet. Als Knoten C habe ich das Kugel-Volumen gewählt, aus welcher die Höhlen-Tunnel zusammen gesetzt werden. Bei Knoten C können die Wege der Höhlen-Tunnel vollständig wiederverwendet werden.

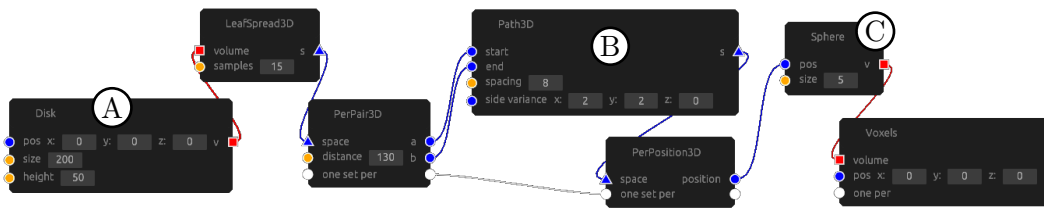
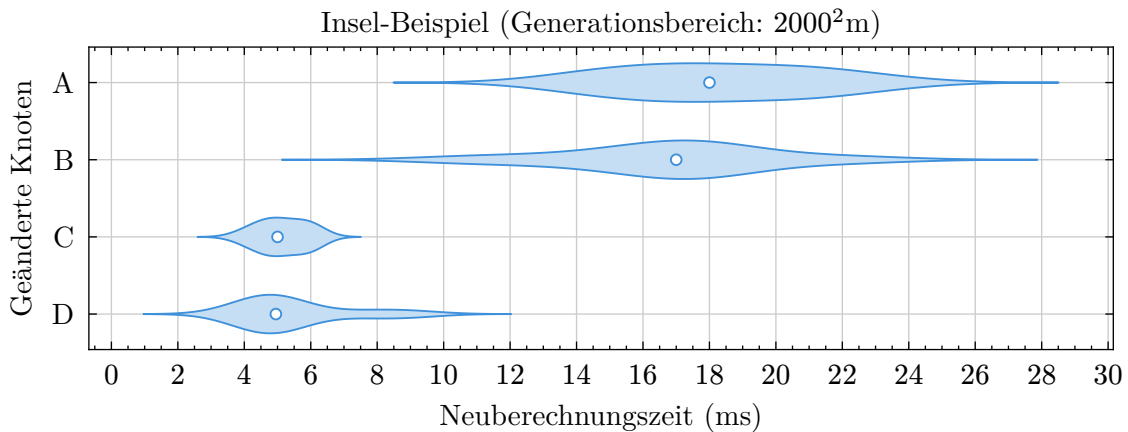


Abbildung 16: Geänderte Knoten im Höhlen-Beispiel-Graph

Aus dem Benchmark ist ersichtlich, dass sich die verschiedenen Neugenerationszeiten nicht stark unterscheiden. Sie skalieren vor allem mit der Menge an Kreuzungen, welcher hier direkten Einfluss auf den in Abschnitt 5.1 besprochenen Branche-Faktor hat. Es ist wahrscheinlich, dass in diesem Benchmark ein Großteil der Berechnungszeit für das erstellen des finalen Volumens benötigt wird.

5.2.2 Insel-Beispiel

Ich habe folgende Knoten im Graphen des Insel-Beispiels für die Benchmarks ausgewählt. Knoten A ist das 2D-Box-Volumen, dass den Generationsbereich definiert. Für B habe ich den Knoten ausgewählt, der die Wege auf den Inseln berechnet. Hier werden die Positionen der Inseln wiederverwendet. C ist das Disk Volumen welches als Boden für die Inseln verwendet wird. D ist das Kugel-Volumen das als Baumkronen verwendet wird. In beiden Fällen werden die Positionen für die Inseln sowie die Positionen für die Bäume und Wege wiederverwendet.



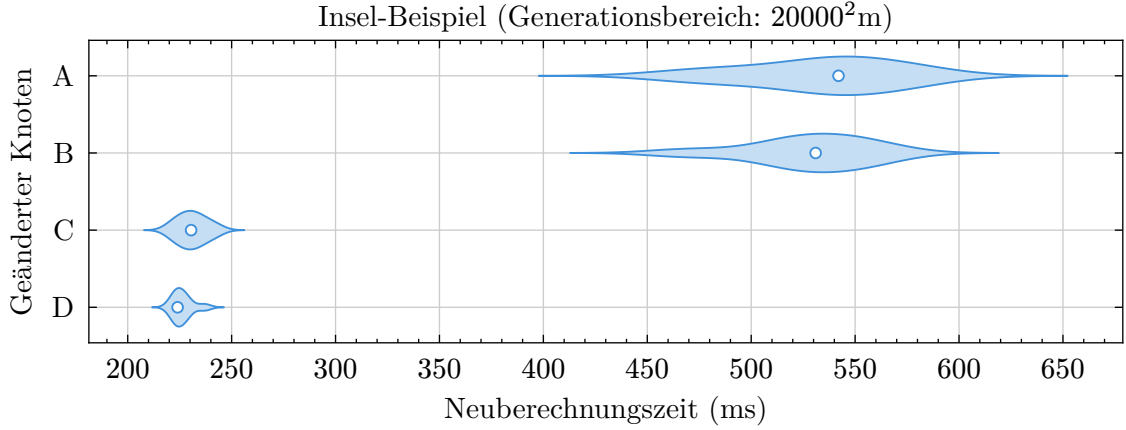


Abbildung 18: Unterschied der Neuberechnungszeit zwischen verschiedenen geänderten Knoten im Insel-Beispiel

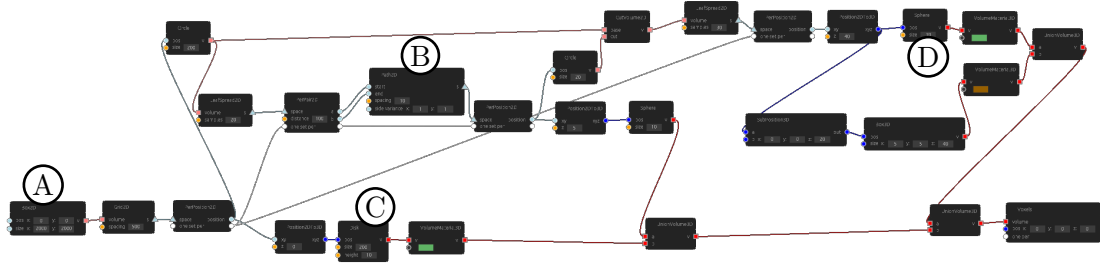


Abbildung 19: Geänderte Knoten im Insel-Beispiel-Graph

In diesen Benchmark sieht man, dass die Neuberechnungszeit nicht linear abfällt, sondern alle Änderungen, wo die Wege und Bäume neu berechnet werden müssen, im Durchschnitt 17ms und 540ms benötigen. Wohin Änderungen, die nur das finale Volumen betreffen, benötigen im Durchschnitt 5ms und 230ms. Somit lässt sich davon ausgehen, dass die Berechnung der Wege und Bäume circa 12ms und 310ms benötigt, welche wegfallen, wenn die zwischengespeicherten Daten verwendet werden.

5.3 Overhead

Ein weiteres wichtiges Kriterium ist der Overhead des Systems. Da der Generationsalgorithmus nicht direkt als kompilierter Programmcode ausgeführt wird, sondern als Abhängigkeits-Graph interpretiert wird, kostet dies zusätzlich Zeit. Diese entsteht insbesondere durch

- die Verwaltung und Modifikation der Graphstruktur
- und durch das rekursive Auflösen der Abhängigkeiten während der Berechnung.

Hierfür wurden zwei weitere Versionen des Höhlen-Beispiels und des Insel-Beispiels implementiert.

In der ersten weiteren Version würden die Verwaltungs-Logik für den Generator-Graph entfernt. Hier wird der Abhängigkeits-Graph direkt evaluiert, ohne Zwischenspeicher anzulegen oder zu verwalten. Diese Änderung reduziert zwar die Menge an Code signifikant, hat aber keine großen Auswirkungen auf die Laufzeit. Dies hat wahrscheinlich folgende Gründe: Die im Zwischenspeicher genutzten Mengen müssen bei der Evaluation

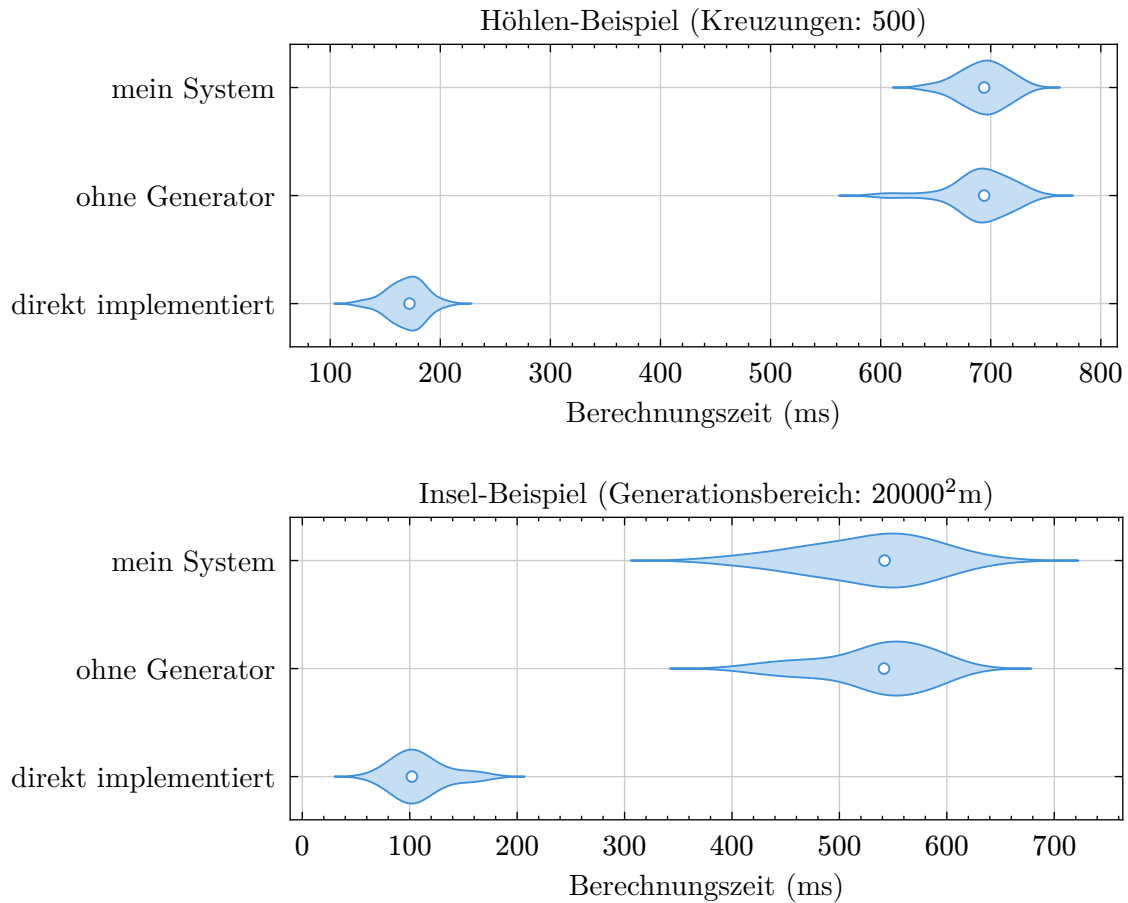


Abbildung 20: Unterschied der Berechnungszeit zwischen meinem System hinzu einer direkten Implementierung

des Graphen ohnehin angelegt werden und werden in diesem Fall nur wieder deallokiert, anstatt weiterhin gespeichert zu bleiben.

Als Zweites wurde eine weitere Version des Generationsalgorithmus ohne Abhängigkeits-Graphen direkt implementiert. Diese Version ist ca. 3-bis 6-mal schneller als das Beispiel. Dies hat wahrscheinlich mit dem Overhead durch die Evaluation des Abhängigkeits-Graphen zu tun. Hier kann durch direkte Schachtelung von Loops die Allokierung von Mengen als Vektoren gespart werden. Dazu benötigt der Abhängigkeits-Graph aufgrund seiner polymorphen Natur viele Switch-Statements, um zwischen den verschiedenen Funktionen, die einen Wert erzeugen, dynamisch zu unterscheiden. Dies fällt bei einer direkten Implementierung weg. Zudem kann der Generationsalgorithmus dadurch stärker durch den Compiler optimiert werden.

5.4 Nutzerfreundlichkeit

Durch den grafischen Editor kann der Generationsprozess als Abhängigkeits-Graph visualisiert und direkt bearbeitet werden. Dadurch lassen sich komplexe Zusammenhänge zwischen einzelnen Operationen leichter nachvollziehen als in klassischem Quellcode, da Datenflüsse und Abhängigkeiten explizit dargestellt sind.

Insbesondere bei der experimentellen Entwicklung von Generationsalgorithmen kann dieser Ansatz Vorteile bieten. Änderungen am Graphen können direkt im Editor vorgenommen werden, ohne dass der gesamte Algorithmus neu implementiert werden muss. In Kombination mit der minimalen Neuberechnung des Generators können Anpassungen am Generationsprozess schneller getestet werden, da nur die betroffenen Teile der Welt neu berechnet werden müssen.

Gleichzeitig hängt die wahrgenommene Nutzerfreundlichkeit stark vom jeweiligen Nutzertypen ab. Nutzer mit viel Erfahrung in klassischer Softwareentwicklung bevorzugen häufig eine direkte Implementierung als Quellcode, da diese mehr Kontrolle bietet und weniger strukturelle Einschränkungen hat. Für weniger erfahrene Nutzer oder für Anwender, die primär konzeptionell arbeiten möchten, kann ein grafischer Editor hingegen zugänglicher sein.

Die visuelle Darstellung der Abhängigkeiten erleichtert das Verständnis des Systems und erlaubt es, Generationslogik zu definieren, ohne sich stark mit den Details einer Programmiersprache beschäftigen zu müssen. Allerdings kann ein solcher Editor auch einschränkend wirken, da nur die im System vorgesehenen Operationen und Strukturen genutzt werden können.

Der grafische Ansatz ist kein Ersatz für klassische Programmierung von prozeduralen Generationsalgorithmen. Stattdessen eröffnet er den Zugang zu einem neuen Nutzerkreis. Wer einen Generationsalgorithmus iterativ entwickeln will und dafür keine umfassende Programmiererfahrung hat, profitiert von der direkten Sichtbarkeit der Abhängigkeiten. Für erfahrene Entwickler dürfte der Editor dagegen eher einschränkend wirken.

5.5 Erweiterbarkeit

Um das System um weitere Operationen oder Datentypen zu erweitern, ist ein gutes Verständnis der Codestruktur erforderlich. Jedoch sind keine grundlegenden Änderungen an der nötig, da der Editor, das Template und der Generator ohne Annahmen zu der Art der Datentypen oder Operationen im Abhängigkeits-Graphen implementiert sind. Die Datentypen und Operationen sind als Typed Unions implementiert, daher können diese einfach erweitert werden. Jedoch muss die neue Variante an jeder Stelle, wo ein Datentyp oder eine Operation allgemein verwendet wird, implementiert werden, um sie in das bestehende Netz an möglichen Abhängigkeiten zu integrieren.

6 Noch offene Fragestellungen

6.1 Nullwerte führen zu leeren Lösungen

Da der Nullwert per Definition ein valider Wert ist, kann es dazu kommen, dass sich ein Abhängigkeitskreis zu Null als Lösung entwickelt, auch wenn es theoretisch andere Lösungen gäbe. Um dies zu lösen, müsste ein anderer Ansatz zur Lösung von Abhängigkeitskreisen genutzt werden. Arbeiten in Richtung Closely Connected Components könnten hier eine Lösung sein.

6.2 Andere Datenstrukturen

Wie schon in Abschnitt 4.7.4 beschrieben eignen sich CSGs nicht zum direkten Rendering. Die Umwandlung zu einem Mesh mit Marching Cubes ist aufwendig und benötigt wesentlich mehr Leistung als die Generation selbst. Somit besteht weiterhin die Forschungsfrage, inwieweit minimale Neuberechnung von prozeduralen Meshes direkt möglich ist.

6.3 Nutzerfreundlichkeitsstudie

Die Nutzerfreundlichkeit des Systems wurde in dieser Arbeit bisher nur qualitativ betrachtet. Eine fundierte Bewertung sollte durch strukturierte Nutzerstudien oder Experteninterviews ergänzt werden. Dabei wäre insbesondere interessant zu erforschen, wie Nutzer mit unterschiedlichem Erfahrungshintergrund mit dem grafischen Editor umgehen.

Erfahrene Nutzer könnten den Editor als zu einschränkend empfinden, da nur die im System vorgesehenen Operationen genutzt werden können, während weniger erfahrene Nutzer von der visuellen Darstellung der Abhängigkeiten profitieren könnten. Eine konkreter Verbesserung der Nutzerfreundlichkeit wäre die Einführung von aussagekräftigen Fehlermeldungen und Warnungen im Editor. Aktuell gibt das System wenig Rückmeldung darüber, warum ein bestimmter Generationsschritt fehlschlägt oder ein unerwartetes Ergebnis liefert.

6.4 Parallelisierung der Generierung

In der aktuellen Implementierung werden unabhängige Knoten im Abhängigkeits-Graphen sequenziell abgearbeitet. Da Knoten desselben Levels keine gegenseitigen Abhängigkeiten haben, könnten diese grundsätzlich parallel berechnet werden. Eine systematische Parallelisierung der levelweisen Berechnung könnte die Generierungszeit bei komplexen Algorithmen mit vielen unabhängigen Operationen erheblich reduzieren. Dabei müsste jedoch sichergestellt werden, dass Schreibzugriffe auf den Generatorgraphen thread-sicher sind, ohne dabei zu viel Overhead durch Synchronisation zu benötigen.

6.5 Automatische Cache-Optimierung

Aktuell muss der Nutzer manuell entscheiden, welche Knoten im Abhängigkeits-Graphen gecacht werden sollen. Ein zu sparsames Caching führt dazu, dass bei Änderungen viele Knoten neu berechnet werden müssen, während ein zu großzügiges Caching unnötig viel Speicher verbraucht und den Overhead erhöht.

Ein interessanter Forschungsansatz wäre ein System, das diese Entscheidungen automatisch trifft. Dafür könnte der Generator die Laufzeit messen, wie häufig ein Knoten wiederverwendet wird und wie aufwendig seine Berechnung ist. Basierend auf diesen Metriken könnte das System dynamisch entscheiden, welche Knoten gecacht werden sollen.

6.6 Skalierung auf sehr große Welten

Die vorliegende Arbeit evaluiert das System anhand vergleichsweise kleiner Beispielen. Es bleibt offen, wie das System mit sehr großen Welten skaliert, die aus tausenden gleichzeitig aktiver Chunks und sehr tiefen Abhängigkeits-Graphen bestehen. Bei sehr großen Welten könnte dies zu erheblichem Speicherbelegung führen. Zukünftige Arbeiten sollten untersuchen, wie ein solches System auf gesamten Spielwelten agiert.

6.7 Integration in bestehende Engines

Diese Arbeit stellt ein System vor, welches in Spiel- oder Simulations-Engines einbettbar sein soll. Jedoch wurde die Frage einer Integration in eine bestehende Game Engine nicht behandelt. Zukünftige Arbeiten sollten untersuchen, welche Schnittstellen eine Engine bereitstellen muss und welcher Integrationsaufwand in bestehende Engines wie Unity oder Unreal Engine entsteht. Relevante Fragestellungen sind insbesondere die Synchronisation zwischen dem Generator und dem Render-Thread sowie die Übergabe extern kontrollierter Variablen wie Kameraposition oder Spielzustand.

6.8 Serialisierung und Persistenz

In der aktuellen Implementierung existieren gecachte Zwischenergebnisse nur im Arbeitsspeicher und gehen beim Beenden der Anwendung verloren. Für einen praktischen Einsatz wäre es jedoch hilfreich, den Zustand des Generatorgraphen auf der Festplatte zu speichern. So könnten bereits berechnete Teile der Welt beim nächsten Start der Anwendung wiederverwendet werden, ohne sie vollständig neu berechnen zu müssen. Dabei stellen sich Fragen zur effizienten Serialisierung großer Graphstrukturen sowie zur Invalidierung gespeicherter Ergebnisse, wenn sich das Template zwischen zwei Sitzungen geändert hat.

7 Fazit

In dieser Arbeit wurde gezeigt, dass minimale Neuberechnung für prozedurale Generierung grundsätzlich möglich ist. Die Beispielimplementierung zeigt, dass sich ein Generationsalgorithmus als Abhängigkeits-Graph darstellen lässt und Zwischenergebnisse gezielt wiederverwendet werden können. Ändert sich ein Teil des Algorithmus, müssen nur die betroffenen Knoten neu berechnet werden.

Da jedoch der Algorithmus als Graph interpretiert wird, anstatt direkt als kompilierter Code ausgeführt zu werden, ist er deutlich langsamer als eine direkte Implementierung. In den Benchmarks war eine optimierte direkte Implementierung etwa 3-bis 6-mal schneller. Dieser Overhead entsteht vor allem durch das rekursive Auflösen der Abhängigkeiten und die polymorphe Natur des Graphen. Hinzu kommt, dass der Aufwand für die Implementierung aller benötigten Operationen und Datentypen hoch ist.

Trotzdem bietet mein Ansatz ein grafisches Interface, um prozedurale Generationsalgorithmen interaktiv zu entwickeln. Es bietet eine vereinfachte Abstraktionsebene zum Testen von Generationslogik, die kein Wissen zu Programmiersprachen benötigt.

Für einen realen Einsatz in einem Spiel oder einer Simulation wäre mein Ansatz sinnvoll, wenn der Generationsalgorithmus für eine prozedurale Welt von Personen entwickelt werden soll, die nicht umfassend programmieren können und Änderungen sehr interaktiv testen wollen.

8 Bibliografie

- Amanatides, John, und Andrew Woo. „A Fast Voxel Traversal Algorithm for Ray Tracing“. *Eurographics*, Spezialausgabe of *Eurographics*, Bd. 87, Nr. 3, 1987, S. 3–10.
- Azmy, Aly. „Weirdest Map Seeds In Minecraft“. *Game Rant*, 28. August 2023, <https://gamerant.com/minecraft-weirdest-seeds>.
- Bazzaz, Mahsa, und Seth Cooper. „Level Generation with Constrained Expressive Range“. *Proceedings of the 20th International Conference on the Foundations of Digital Games*, Spezialausgabe of *Proceedings of the 20th International Conference on the Foundations of Digital Games*, FDG '25, 2025, <https://doi.org/10.1145/3723498.3723845>.
- Blender Foundation, und Blender Institute. „Blender: Free and Open Source 3D Creation Suite“. Blender Foundation, 2026, <https://www.blender.org/>.
- Bollobás, Béla. *Modern Graph Theory*. 1. Aufl., Bd. 184, Springer, 1998, S. XIV, 394, <https://doi.org/10.1007/978-1-4612-0619-4>. Graduate Texts in Mathematics.
- Coster, Tim. „Unity Shader Graph Custom Function Nodes Collection“. *Coster-Graphics*, 7. September 2020, <https://timcoster.com/2020/09/07/unity-shader-graph-custom-function-nodes-collection/>.
- Glavina, Stjepan. „async-channel: Async Multi-Producer Multi-Consumer Channel“. 2025, <https://crates.io/crates/async-channel>.
- Goslin, Alexander. „InfiniteDiffusion: Bridging Learned Fidelity and Procedural Utility for Open-World Terrain Generation“. *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, Spezialausgabe of *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers* [New York, NY, USA], SIGGRAPH Conference Papers '26, 2026, <https://doi.org/10.48550/arXiv.2512.08309>.
- Hoare, C. A. R. „Communicating Sequential Processes“. *Communications of the ACM* [New York, NY, USA], Bd. 21, Nr. 8, August 1978, S. 666–77, <https://doi.org/10.1145/359576.359585>.
- Laine, Samuli, und Tero Karras. „Efficient Sparse Voxel Octrees“. *IEEE Transactions on Visualization and Computer Graphics* [Los Alamitos, CA, USA], Bd. 17, Nr. 8, August 2011, S. 1048–59, <https://doi.org/10.1109/TVCG.2010.240>.
- Lorensen, William E., und Harvey E. Cline. „Marching cubes: A high resolution 3D surface construction algorithm“. *SIGGRAPH Comput. Graph.* [New York, NY, USA], Bd. 21, Nr. 4, August 1987, S. 163–69, <https://doi.org/10.1145/37402.37422>.
- Merrell, Paul. „Example-based model synthesis“. *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*, Spezialausgabe of *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games* [Seattle, Washington], I3D '07, 2007, S. 105–12, <https://doi.org/10.1145/1230100.1230119>.

- Merrell, Paul. „Example-Based Procedural Modeling Using Graph Grammars“. *ACM Trans. Graph.* [New York, NY, USA], Bd. 42, Nr. 4, Juli 2023, <https://doi.org/10.1145/3592119>.
- Peters, Orson. „slotmap: Slotmap Data Structure for Rust“. 2024, <https://crates.io/crates/slotmap>.
- Prusinkiewicz, Przemyslaw, und Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. Springer-Verlag, 1990, <https://doi.org/10.1007/978-1-4613-8476-2>. The Virtual Laboratory.
- Requicha, Aristides A. G., und Herbert B. Voelcker. *Constructive Solid Geometry*. Technical Memorandum no. TM-25, Rochester, NY, USA, November 1977.
- Scott, David W. *Multivariate Density Estimation: Theory, Practice, and Visualization*. 1. Aufl., John Wiley & Sons, 1992, <https://doi.org/10.1002/9780470316849>. Wiley Series in Probability and Mathematical Statistics.
- The Servo Project Developers. „smallvec: \"Small vector\" optimization for Rust: store up to a small number of items on the stack“. 2024, <https://crates.io/crates/smallvec>.
- Side Effects Software Inc. „Houdini: 3D Procedural Software for Modeling, Animation, VFX, Look Development, Lighting and Rendering“. Side Effects Software Inc., 2026, <https://www.sidefx.com/>.
- Solkoll. „L-system trees form realistic models of natural patterns“. 2025, https://en.wikipedia.org/wiki/L-system#/media/File:Dragon_trees.jpg.
- Volz, Vanessa, u. a. „Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network“. 2018, <https://doi.org/10.48550/arXiv.1805.00728>.
- Wassermann, Benjamin, u. a. „From geometric design to numerical analysis: A direct approach using the Finite Cell Method on Constructive Solid Geometry“. *Computers & Mathematics with Applications*, Bd. 74, Nr. 7, Oktober 2017, S. 1703–26, <https://doi.org/10.1016/j.camwa.2017.01.027>.
- Withington, Oliver, und Laurissa Tokarchuk. „Compressing and Comparing the Generative Spaces of Procedural Content Generators“. *2022 IEEE Conference on Games (CoG)*, Spezialausgabe of *2022 IEEE Conference on Games (CoG)*, Nr. , 2022, S. 143–50, <https://doi.org/10.1109/CoG51982.2022.9893615>.
- Alle Web-Verweise wurden am 01.06.2026 zuletzt aufgerufen.

Eigenständigkeits- und Einverständniserklärung zur Überprüfung mit Plagiatsoftware sowie die Erklärung zur Veröffentlichung bei Bachelor- und Masterarbeiten

Declarations of Authorship and Consent for Checking with Plagiarism Software and the Declaration of Publication for Bachelor's and Master's Thesis

Studierenden-Angaben / Student Information:

Matrikelnr. / Student ID 6189259

Nachname / Surname Behn

Vorname / First Name Maarten

Titel der Arbeit / Title of Thesis

Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen
Welten-Generation.

A) Eigenständigkeitserklärung / Declaration of Authorship

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet, dazu zählen auch KI-basierte Anwendungen oder Werkzeuge.
Die Arbeit wurde in gleicher oder ähnlicher Form noch nicht als Prüfungsleistung eingereicht.

I hereby affirm that I have written the present work independently and have used no sources or aids other than those indicated. All parts of my work that have been taken from other works, either verbatim or in terms of meaning, have been marked as such, indicating the source. The same applies to drawings, sketches, pictorial representations and sources from the Internet, including AI-based applications or tools. The work has not yet been submitted in the same or a similar form as a final examination paper.

- ☒ Ich habe KI-basierte Anwendungen und/oder Werkzeuge genutzt und diese im Anhang "Nutzung KI basierte Anwendungen" dokumentiert.

I have used AI-based applications and/or tools and documented them in the appendix "Use of AI-based applications".

B) Erklärung zur Veröffentlichung von Bachelor- oder Masterarbeiten

Declaration regarding the publication of bachelor's or master's thesis

Die Abschlussarbeit wird zwei Jahre nach Studienabschluss dem Archiv der Universität Bremen zur dauerhaften Archivierung angeboten. Archiviert werden:

Two years after graduation, the thesis is offered to the archive of the University of Bremen for permanent archiving. The following are archived:

- 1) Masterarbeiten mit lokalem oder regionalem Bezug sowie pro Studienfach und Studienjahr 10 % aller Masterarbeiten
Master's theses with a local or regional focus, as well as per subject and academic year 10% of all Master's thesis
- 2) Bachelorarbeiten des jeweils ersten und letzten Bachelorabschlusses pro Studienfach und Jahr.
Bachelor's thesis for the first and last bachelor's degrees per subject and year.

☒ Ich bin damit einverstanden, dass meine Abschlussarbeit im Universitätsarchiv für wissenschaftliche Zwecke von Dritten eingesehen werden darf.
I agree that my thesis may be viewed by third parties in the university archive for academic purposes.

☒ Ich bin damit einverstanden, dass meine Abschlussarbeit nach 30 Jahren (gem. §7 Abs. 2 BremArchivG) im Universitätsarchiv für wissenschaftliche Zwecke von Dritten eingesehen werden darf.
I agree that my thesis may be viewed by third parties for academic purposes in the university archive after 30 years (in accordance with §7 para. 2 BremArchivG).

☐ Ich bin **nicht** damit einverstanden, dass meine Abschlussarbeit im Universitätsarchiv für wissenschaftliche Zwecke von Dritten eingesehen werden darf.
I do not consent to my thesis being made available in the university archive for third parties to view for academic purposes.

C) Einverständniserklärung zur elektronischen Überprüfung der Arbeit auf Plagiate Declaration of consent for electronic checking of the work for plagiarism

Eingereichte Arbeiten können nach § 18 des Allgemeinen Teil der Bachelor- bzw. der Masterprüfungsordnungen der Universität Bremen mit qualifizierter Software auf Plagiatsvorwürfe untersucht werden.

Zum Zweck der Überprüfung auf Plagiate erfolgt das Hochladen auf den Server der von der Universität Bremen aktuell genutzten Plagiatssoftware.

Submitted papers can be checked for plagiarism using qualified software in accordance with § 18 of the General Section of the Bachelor's or Master's Degree Examination Regulations of the University of Bremen. For the purpose of checking for plagiarism, the upload to the server is done using the plagiarism software currently used by the University of Bremen.

- ☒ Ich bin damit einverstanden, dass die von mir vorgelegte und verfasste Arbeit zum oben genannten Zweck dauerhaft auf dem externen Server der aktuell von der Universität Bremen genutzten Plagiatssoftware, in einer institutionseigenen Bibliothek (Zugriff nur durch die Universität Bremen), gespeichert wird.

I agree that the work I have submitted and written will be stored permanently on the external server of the plagiarism software currently used by the University of Bremen, in a library belonging to the institution (accessed only by the University of Bremen), for the above-mentioned purpose.

- ☐ Ich bin **nicht** damit einverstanden, dass die von mir vorgelegte und verfasste Arbeit zum o.g. Zweck dauerhaft auf dem externen Server der aktuell von der Universität Bremen genutzten Plagiatssoftware, in einer institutionseigenen Bibliothek (Zugriff nur durch die Universität Bremen), gespeichert wird.

I do not consent to the work I submitted and wrote being permanently stored on the external server of the plagiarism software currently used by the University of Bremen, in a library belonging to the institution (accessed only by the University of Bremen), for the above-mentioned purpose.

Das Einverständnis der dauerhaften Speicherung des Textes ist freiwillig. Die Einwilligung kann jederzeit durch Erklärung gegenüber der Universität Bremen, mit Wirkung für die Zukunft, widerrufen werden. Weitere Informationen zur Überprüfung von schriftlichen Arbeiten durch die Plagiatsoftware sind im Nutzungs- und Datenschutzkonzept enthalten. Diese finden Sie auf der Internetseite der Universität Bremen.

Consent to the permanent storage of the text is voluntary. Consent can be withdrawn at any time by making a declaration to this effect to the University of Bremen, with effect for the future. Further information on the checking of written work using plagiarism software can be found in the data protection and usage concept. This can be found on the University of Bremen website.

Mit meiner Unterschrift versichere ich, dass ich die obenstehenden Erklärungen gelesen und verstanden habe und bestätige die Richtigkeit der gemachten Angaben.

With my signature, I confirm that I have read and understood the above explanations and confirm the accuracy of the information provided.

Datum / Date

Unterschrift/ Signature

Nutzung KI basierte Anwendungen

Neuronale Large Language Modelle wurden zur Erstellung dieser Arbeit in folgenden Bereichen verwendet:

Recherche

Um einen Überblick über den Stand der Technik zu erhalten und relevante Arbeiten zu identifizieren, wurden KI-basierte Systeme genutzt, um Empfehlungen für bestehende Literatur zu generieren.

Es wurden folgende Prompts genutzt:

- „Ich habe für meine Bachelor Arbeit folgende Idee <Idee>. Suche wissenschaftlichen Arbeiten, die sich mit der Idee beschäftigen.“
- „Konzept <X> finde ich spannend. Suche nach wissenschaftlichen Arbeiten, die dies tiefer in Richtung <Y> erweitern.“

Rechtschreib- und Satzbaukorrektur

Eine von mir verfasste Rohfassung wurde mithilfe von KI auf Rechtschreibung, Grammatik und Zeichensetzung überprüft. Dabei wurde die KI gezielt so eingesetzt, dass nur notwendige Korrekturen vorgenommen wurden. Stilistische Änderungen wurden vermieden, um den ursprünglichen Ausdruck beizubehalten. Es wurde darauf geachtet, dass sich der Inhalt durch die vorgeschlagenen Anpassungen nicht ändert.

Es wurden folgende Prompts genutzt:

- „<Textausschnitt> gehe diesen Text sorgfältig durch. Und liste alle Rechtschreib- und Satzbaufehler auf. Verändere dabei den Inhalt oder die Aussagen nicht. Gib Verbesserungsvorschläge an.“

Genutzte Tools

- Chat GPT: <https://chatgpt.com> Zuletzt zugegriffen am 29.05.2026
- Claude: <https://claude.ai> Zuletzt zugegriffen am 29.05.2026

Bereiche, in denen keine KI verwendet wurde, sind:

- Programmierung: Es wurde keine KI verwendet, um den Code meiner Implementierung zu schreiben.
- Textinhalte: KI wurde nicht verwendet, um inhaltliche Aussagen, Argumentationen oder Ergebnisse der Arbeit zu generieren.